



सर्व विद्या अधिपतये

Faculty of Engineering (Diploma)

Semester – 5

Subject Name – Introduction to Machine Learning

Subject Code – DE103501

Faculty Name – Ashwini Walake

Unit -1: Introduction to machine learning

Overview of Human Learning and Machine Learning

Human Learning

Human learning is the process by which people acquire knowledge, skills, attitudes, and experience through observation, study, practice, and interaction with the environment. Humans learn continuously and can apply reasoning, creativity, emotions, and common sense to solve problems.

Characteristics of Human Learning

- Learns from experience and observation.
- Uses reasoning, intuition, and emotions.
- Can adapt to new situations with limited examples.
- Learns throughout life.
- Can transfer knowledge from one domain to another.
- Makes decisions using common sense and judgment.

Example

A child learns to identify a cat by seeing different cats, listening to others, and interacting with them. Even after seeing only a few examples, the child can recognize new cats.

Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn from data and improve their performance without being explicitly programmed for every task.

Instead of following fixed instructions, machine learning algorithms identify patterns in data, make predictions, and improve accuracy as more data becomes available.

Characteristics of Machine Learning

- Learns from large amounts of data.
- Improves performance with experience (training).
- Makes predictions and classifications.
- Requires algorithms and computational models.
- Does not possess emotions or common sense.
- Performance depends on the quality and quantity of training data.

Example

An email spam filter learns from thousands of emails. After training, it automatically classifies new emails as spam or not spam.

Difference Between Human Learning and Machine Learning

Human Learning	Machine Learning
Learns from experience, observation, and education.	Learns from data using algorithms.
Uses intelligence, emotions, and common sense.	Uses mathematical models and statistical methods.
Can learn from very few examples.	Usually requires a large amount of data.
Learns continuously throughout life.	Learns during the training process and can be retrained.
Can think creatively and logically.	Cannot think creatively; follows learned patterns.
Makes decisions based on reasoning and experience.	Makes predictions based on learned data patterns.
Can understand context and emotions.	Cannot understand emotions or real-world context naturally.
Learning speed is relatively slow.	Can process and learn from huge datasets quickly.
Can generalize knowledge across different domains.	Usually specializes in the task it was trained for.
Example: A student learning mathematics.	Example: A recommendation system suggesting movies.

Machine Learning Life Cycle:

Machine learning life cycle involves seven major steps, which are given below:

1. Gathering Data
2. Data preparation
3. Data Wrangling
4. Analyze Data
5. Train the model
6. Test the model
7. Deployment

1. Gathering Data:

- In this step, we need to identify the different data sources, as data can be collected from various sources such as files, database, internet, or mobile devices. It is one of the most important steps of the life cycle. The quantity and quality of the collected data will determine the efficiency of the output. The more will be the data, the more accurate will be the prediction.

This step includes the below tasks:

- Identify various data sources
- Collect data
- Integrate the data obtained from different sources

2. Data preparation:

In this step, first, we put all data together, and then randomize the ordering of data.

This step can be further divided into two processes:

Data exploration:

- It is used to understand the nature of data that we have to work with. We need to understand the characteristics, format, and quality of data.
- A better understanding of data leads to an effective outcome. In this, we find Correlations, general trends, and outliers.

Data pre-processing:

- Now the next step is preprocessing of data for its analysis.

3. Data Wrangling:

- Data wrangling is the process of cleaning and converting raw data into a useable format.
- It is the process of cleaning the data, selecting the variable to use, and transforming the data in a proper format to make it more suitable for analysis in the next step.
- It is one of the most important steps of the complete process. Cleaning of data is required to address the quality issues.
- It is not necessary that data we have collected is always of our use as some of the data may not be useful.

In real-world applications, collected data may have various issues, including:

- **Missing Values**
- **Duplicate data**
- **Invalid data**
-

4. Analyse Data:

- Now the cleaned and prepared data is passed on to the analysis step. This step involves:
- Selection of analytical techniques
- Building models
- Review the result
- The aim of this step is to build a machine learning model to analyze the data using various analytical techniques and review the outcome.
- It starts with the determination of the type of the problems, where we select the machine learning techniques such as Classification, Regression, Cluster analysis, Association, etc. then build the model using prepared data, and evaluate the model

5. Train the model:

- Now the next step is to train the model, in this step we train our model to improve its performance for better outcome of the problem.
- We use datasets to train the model using various machine learning algorithms.

6. Test the model:

- Once our machine learning model has been trained on a given dataset, then we test the model. In this step, we check for the accuracy of our model by providing a test dataset to it.

7. Deployment:

- The last step of machine learning life cycle is deployment, where we deploy the model in the real-world system

Types of Machine Learning

1. Supervised Machine Learning

Imagine a teacher supervising a class. The teacher already knows the correct answers but the learning process doesn't stop until the students learn the answers as well. This is the essence of Supervised Machine Learning Algorithms. Here, the algorithm learns from a training dataset and makes predictions that are compared with the actual output values. If the predictions are not correct, then the algorithm is modified until it is satisfactory. This learning process continues until the algorithm achieves the required level of performance. Then it can provide the desired output values for any new inputs.

2. Unsupervised Machine Learning

In this case, there is no teacher for the class and the students are left to learn for themselves! So for Unsupervised Machine Learning Algorithms, there is no specific answer to be learned and there is no teacher. In this way, the algorithm doesn't figure out any output for input but it explores the data. The algorithm is left unsupervised to find the underlying structure in the data in order to learn more and more about the data itself.

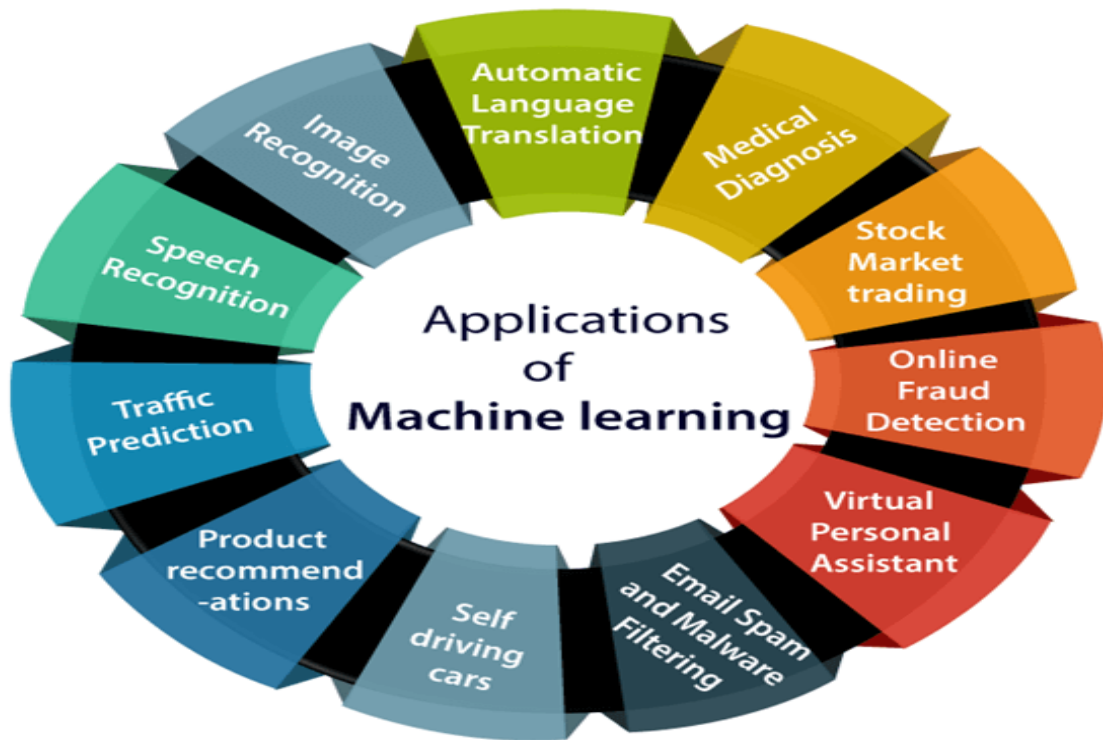
3. Reinforcement Learning.

Well, here are the hypothetical students who learn from their own mistakes over time (that's like life!). So the Reinforcement Machine Learning Algorithms learn optimal actions through trial and error. This means that the algorithm decides the next action by learning behaviors that are based on its current state and that will maximize the reward in the future. This is done using reward feedback that allows the Reinforcement Algorithm to learn which the best behaviors that lead to maximum reward are. This reward feedback is known as a reinforcement signal.

Applications of Machine Learning

Applications of Machine learning

Machine learning is a buzzword for today's technology, and it is growing very rapidly day by day. We are using machine learning in our daily life even without knowing it such as Google Maps, Google assistant, Alexa, etc. Below are some most trending real-world applications of Machine Learning:



1. Image Recognition:

Image recognition is one of the most common applications of machine learning. It is used to identify objects, persons, places, digital images, etc. The popular use case of image recognition and face detection is, **Automatic friend tagging suggestion**:

Facebook provides us a feature of auto friend tagging suggestion. Whenever we upload a photo with our Facebook friends, then we automatically get a tagging suggestion with name, and the technology behind this is machine learning's **face detection** and **recognition algorithm**.

It is based on the Facebook project named "**Deep Face**," which is responsible for face recognition and person identification in the picture.

2. Speech Recognition

While using Google, we get an option of "**Search by voice**," it comes under speech recognition, and it's a popular application of machine learning.

Speech recognition is a process of converting voice instructions into text, and it is also known as "**Speech to text**", or "**Computer speech recognition**." At present, machine learning algorithms are widely used by various applications of speech recognition. **Google**

assistant, Siri, Cortana, and Alexa are using speech recognition technology to follow the voice instructions.

3. Traffic prediction:

If we want to visit a new place, we take help of Google Maps, which shows us the correct path with the shortest route and predicts the traffic conditions.

It predicts the traffic conditions such as whether traffic is cleared, slow-moving, or heavily congested with the help of two ways:

- **Real Time location** of the vehicle from Google Map app and sensors
- **Average time has taken** on past days at the same time.

Everyone who is using Google Map is helping this app to make it better. It takes information from the user and sends back to its database to improve the performance.

4. Product recommendations:

Machine learning is widely used by various e-commerce and entertainment companies such as **Amazon, Netflix**, etc., for product recommendation to the user. Whenever we search for some product on Amazon, then we started getting an advertisement for the same product while internet surfing on the same browser and this is because of machine learning.

Google understands the user interest using various machine learning algorithms and suggests the product as per customer interest.

As similar, when we use Netflix, we find some recommendations for entertainment series, movies, etc., and this is also done with the help of machine learning.

5. Self-driving cars:

One of the most exciting applications of machine learning is self-driving cars. Machine learning plays a significant role in self-driving cars. Tesla, the most popular car manufacturing company is working on self-driving car. It is using unsupervised learning method to train the car models to detect people and objects while driving.

6. Email Spam and Malware Filtering:

Whenever we receive a new email, it is filtered automatically as important, normal, and spam. We always receive an important mail in our inbox with the important symbol and spam emails in our spam box, and the technology behind this is Machine learning. Below are some spam filters used by Gmail:

- Content Filter
- Header filter
- General blacklists filter

- Rules-based filters
- Permission filters

Some machine learning algorithms such as **Multi-Layer Perceptron**, **Decision tree**, and **Naïve Bayes classifier** are used for email spam filtering and malware detection.

7. Virtual Personal Assistant:

We have various virtual personal assistants such as **Google assistant**, **Alexa**, **Cortana**, **Siri**. As the name suggests, they help us in finding the information using our voice instruction. These assistants can help us in various ways just by our voice instructions such as Play music, call someone, Open an email, Scheduling an appointment, etc.

These virtual assistants use machine learning algorithms as an important part.

These assistant record our voice instructions, send it over the server on a cloud, and decode it using ML algorithms and act accordingly.

8. Online Fraud Detection:

Machine learning is making our online transaction safe and secure by detecting fraud transaction. Whenever we perform some online transaction, there may be various ways that a fraudulent transaction can take place such as **fake accounts**, **fake ids**, and **steal money** in the middle of a transaction. So to detect this, **Feed Forward Neural network** helps us by checking whether it is a genuine transaction or a fraud transaction.

For each genuine transaction, the output is converted into some hash values, and these values become the input for the next round. For each genuine transaction, there is a specific pattern which gets change for the fraud transaction hence, it detects it and makes our online transactions more secure.

9. Stock Market trading:

Machine learning is widely used in stock market trading. In the stock market, there is always a risk of up and downs in shares, so for this machine learning's **long short term memory neural network** is used for the prediction of stock market trends.

10. Medical Diagnosis:

In medical science, machine learning is used for diseases diagnoses. With this, medical technology is growing very fast and able to build 3D models that can predict the exact position of lesions in the brain.

It helps in finding brain tumors and other brain-related diseases easily.

11. Automatic Language Translation:

Nowadays, if we visit a new place and we are not aware of the language then it is not a problem at all, as for this also machine learning helps us by converting the text into our known languages. Google's GNMT (Google Neural Machine Translation) provide this feature, which is a Neural Machine Learning that translates the text into our familiar language, and it called as automatic translation.

The technology behind the automatic translation is a sequence to sequence learning algorithm, which is used with image recognition and translates the text from one language to another language.

Tools and Technology for Machine Learning

Machine Learning Tools

Machine learning is one of the most revolutionary technologies that is making lives simpler. **It is a subfield of Artificial Intelligence, which analyses the data, build the model, and make predictions.** Due to its popularity and great applications, every tech enthusiast wants to learn and build new machine learning Apps. However, to build ML models, it is important to master machine learning tools. Mastering machine learning tools will enable you to play with the data, train your models, discover new methods, and create algorithms.

There are different tools, software, and platform available for machine learning, and also new software and tools are evolving day by day. Although there are many options and availability of Machine learning tools, choosing the best tool per your model is a challenging task. If you choose the right tool for your model, you can make it faster and more efficient. In this topic, we will discuss some popular and commonly used Machine learning tools and their features.

1. Tensor Flow



TensorFlow is one of the most popular open-source libraries used to train and build both machine learning and deep learning models. It provides a JS library and was developed by **Google Brain Team**. It is much popular among machine learning enthusiasts, and they use it for building different ML applications. It offers a powerful library, tools, and resources for numerical computation, specifically for large scale machine learning and deep learning projects. It enables data scientists/ML developers to build and deploy machine learning applications efficiently. For training and building the ML models, TensorFlow provides a high-level Keras API, which lets users easily start with TensorFlow and machine learning.

Features:

Below are some top features:

- TensorFlow enables us to build and train our ML models easily.
- It also enables you to run the existing models using **the TensorFlow.js**
- It provides multiple abstraction levels that allow the user to select the correct resource as per the requirement.
- It helps in building a neural network.
- Provides support of distributed computing.
- While building a model, for more need of flexibility, it provides eager execution that enables immediate iteration and intuitive debugging.
- This is open-source software and highly flexible.
- It also enables the developers to perform numerical computations using data flow graphs.
- Run-on GPUs and CPUs, and also on various mobile computing platforms.
- It provides a functionality of auto diff (Automatically computing gradients is called automatic differentiation or auto diff).
- It enables to easily deploy and training the model in the cloud.
- It can be used in two ways, i.e., by installing through NPM or by script tags.
- It is free to use.

2. PyTorch



PyTorch is an open-source machine learning framework, which is based on **the Torch** library. This framework is free and open-source and developed by **FAIR(Facebook's AI Research lab)**. It is one of the popular ML frameworks, which can be used for various applications, including computer vision and natural language processing. PyTorch has Python and C++ interfaces; however, the Python interface is more interactive. Different deep learning software is made up on top of PyTorch, such as PyTorch Lightning, Hugging Face's Transformers, Tesla autopilot, etc.

It specifies a Tensor class containing an n-dimensional array that can perform tensor computations along with GPU support.

Features:

Below are some top features:

- It enables the developers to create neural networks using Autograde Module.
- It is more suitable for deep learning researches with good speed and flexibility.
- It can also be used on cloud platforms.
- It includes tutorial courses, various tools, and libraries.
- It also provides a dynamic computational graph that makes this library more popular.
- It allows changing the network behaviour randomly without any lag.
- It is easy to use due to its hybrid front-end.
- It is freely available.

3. Google Cloud ML Engine



While training a classifier with a huge amount of data, a computer system might not perform well. However, various machine learning or deep learning projects require millions or billions of training datasets. Or the algorithm that is being used is taking a long time for execution. In such a case, one should go for the Google Cloud ML Engine. It is a hosted platform where ML developers and data scientists build and run optimum quality machine learning models. It provides a managed service that allows developers to easily create ML models with any type of data and of any size.

Features:

Below are the top features:

- Provides machine learning model training, building, deep learning and predictive modelling.
- The two services, namely, prediction and training, can be used independently or combinedly.
- It can be used by enterprises, i.e., for identifying clouds in a satellite image, responding faster to emails of customers.
- It can be widely used to train a complex model.

4. Amazon Machine Learning (AML)



Amazon provides a great number of machine learning tools, and one of them is **Amazon Machine Learning** or AML. Amazon Machine Learning (AML) is a cloud-based and robust machine learning software application, which is widely used for building machine learning models and making predictions. Moreover, it integrates data from multiple sources, including **Redshift, Amazon S3, or RDS**.

Features

Below are some top features:

- AML offers visualization tools and wizards.
- Enables the users to identify the patterns, build mathematical models, and make predictions.
- It provides support for three types of models, which are multi-class classification, binary classification, and regression.
- It permits users to import the model into or export the model out from Amazon Machine Learning.
- It also provides core concepts of machine learning, including ML models, Data sources, Evaluations, Real-time predictions and Batch predictions.
- It enables the user to retrieve predictions with the help of batch APIs for bulk requests or real-time APIs for individual requests.

5. NET



Accord.Net is .Net based Machine Learning framework, which is used for scientific computing. It is combined with audio and image processing libraries that are written in C#. This framework provides different libraries for various applications in ML, such as **Pattern Recognition, linear algebra, Statistical Data processing**. One popular package of the Accord.Net framework is **Accord. Statistics, Accord.Math, and Accord.MachineLearning**.

Features

Below are some top features:

- It contains 38+ kernel Functions.
- Consists of more than 40 non-parametric and parametric estimation of statistical distributions.
- Used for creating production-grade computer audition, computer vision, signal processing, and statistics apps.
- Contains more than 35 hypothesis tests that include two-way and one way ANOVA tests, non-parametric tests such as the Kolmogorov-Smirnov test and many more.

6. Apache Mahout

Apache Mahout is an open-source project of Apache Software Foundation, which is used for developing machine learning applications mainly focused on Linear Algebra. It is a distributed linear algebra framework and mathematically expressive Scala DSL, which enable the developers to promptly implement their own algorithms. It also provides Java/Scala libraries to perform Mathematical operations mainly based on linear algebra and statistics.

Features:

Below are some top features:

- It enables developers to implement machine learning techniques, including recommendation, clustering, and classification.
- It is an efficient framework for implementing scalable algorithms.
- It consists of matrix and vector libraries.
- It provides support for multiple distributed backends(including Apache Spark)
- It runs on top of Apache Hadoop using the MapReduce paradigm.

7. Shogun



Shogun is a free and open-source machine learning software library, which was created by **Gunnar Raetsch and Soeren Sonnenburg** in the year **1999**. This software library is written in C++ and supports interfaces for different languages such as Python, R, Scala, C#, Ruby, etc., using **SWIG**(Simplified Wrapper and Interface Generator). The main aim of Shogun is on different kernel-based algorithms such as Support Vector Machine (SVM), K-

Means Clustering, etc., for regression and classification problems. It also provides the complete implementation of Hidden Markov Models.

Features:

Below are some top features:

- The main aim of Shogun is on different kernel-based algorithms such as Support Vector Machine (SVM), K-Means Clustering, etc., for regression and classification problems.
- It provides support for the use of pre-calculated kernels.
- It also offers to use a combined kernel using Multiple kernel Learning Functionality.
- This was initially designed for processing a huge dataset that consists of up to 10 million samples.
- It also enables users to work on interfaces on different programming languages such as Lua, Python, Java, C#, Octave, Ruby, MATLAB, and R.

8. Oryx2



It is a realization of the lambda architecture and built on **Apache Kafka** and **Apache Spark**. It is widely used for real-time large-scale machine learning projects. It is a framework for building apps, including end-to-end applications for filtering, packaged, regression, classification, and clustering. It is written in Java languages, including Apache Spark, Hadoop, Tomcat, Kafka, etc. The latest version of Oryx2 is Oryx 2.8.0.

Features:

Below are some top features:

- It has three tiers: specialization on top providing ML abstractions, generic lambda architecture tier, end-to-end implementation of the same standard ML algorithms.
- The original project of Oryx2 was Oryx1, and after some upgrades, Oryx2 was launched.
- It is well suited for large-scale real-time machine learning projects.
- It contains three layers which are arranged side-by-side, and these are named as Speed layer, batch layer, and serving layer.

- It also has a data transport layer that transfer data between different layers and receives input from external sources.

9. Apache Spark MLlib



Apache Spark MLlib is a scalable machine learning library that runs on Apache Mesos, Hadoop, Kubernetes, standalone, or in the cloud. Moreover, it can access data from different data sources. It is an open-source cluster-computing framework that offers an interface for complete clusters along with data parallelism and fault tolerance.

For optimized numerical processing of data, MLlib provides linear algebra packages such as Breeze and netlib-Java. It uses a query optimizer and physical execution engine for achieving high performance with both batch and streaming data.

Features

Below are some top features:

- MLlib contains various algorithms, including Classification, Regression, Clustering, recommendations, association rules, etc.
- It runs different platforms such as Hadoop, Apache Mesos, Kubernetes, standalone, or in the cloud against diverse data sources.
- It contains high-quality algorithms that provide great results and performance.
- It is easy to use as it provides interfaces In Java, Python, Scala, R, and SQL.

10. Google ML kit for Mobile



For Mobile app developers, Google brings ML Kit, which is packaged with the expertise of machine learning and technology to create more robust, optimized, and personalized apps. This tools kit can be used for face detection, text recognition, landmark detection, image labelling, and barcode scanning applications. One can also use it for working offline.

Features:

Below are some top features:

- The ML kit is optimized for mobile.

- It includes the advantages of different machine learning technologies.
- It provides easy-to-use APIs that enables powerful use cases in your mobile apps.
- It includes Vision API and Natural Language APIS to detect faces, text, and objects, and identify different languages & provide reply suggestions.

UNIT 2: Python libraries suitable for Machine Learning

Basics of NumPy Arrays

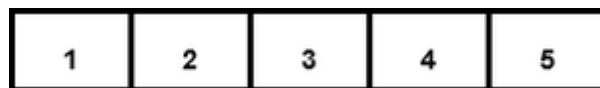
NumPy stands for Numerical Python. It is a Python library used for working with an array. In Python, we use the list for purpose of the array but it's slow to process. NumPy array is a powerful N-dimensional array object and its use in linear algebra, Fourier transform, and random number capabilities. It provides an array object much faster than traditional Python lists.

Types of Array:

1. One Dimensional Array
2. Multi-Dimensional Array

One Dimensional Array:

A one-dimensional array is a type of linear array.



One Dimensional Array

Example:

```
# importing numpy module
import numpy as np

# creating list
list = [1, 2, 3, 4]

# creating numpy array
sample_array = np.array(list)

print("List in python : ", list)

print("Numpy Array in python :",
      sample_array)
```

Output:

List in python: [1, 2, 3, 4]

Numpy Array in python: [1 2 3 4]

Check data type for list and array:

```
print(type(list_1))
```

```
print(type(sample_array))
```

Output:

<class 'list'>

<class 'numpy.ndarray'>

Multi-Dimensional Array:

Data in multidimensional arrays are stored in tabular form.

1	2	3	4	5
6	7	8	9	10

Two Dimensional Array Example:

```
# importing numpy module
import numpy as np

# creating list
list_1 = [1, 2, 3, 4]
list_2 = [5, 6, 7, 8]
list_3 = [9, 10, 11, 12]
# creating numpy array
sample_array = np.array([list_1,
                          list_2,
                          list_3])
print("Numpy multi dimensional array in python\n",
      sample_array)
```

Output:

Numpy multi dimensional array in python

```
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
```

Note: use [] operators inside numpy.array() for multi-dimensional

Anatomy of an array:

1. Axis: The Axis of an array describes the order of the indexing into the array.

Axis 0 = one dimensional

Axis 1 = Two dimensional

Axis 2 = Three dimensional

2. Shape: The number of elements along with each axis. It is from a tuple.

Example:

```
# importing numpy module
import numpy as np

# creating list
list_1 = [1, 2, 3, 4]
list_2 = [5, 6, 7, 8]
list_3 = [9, 10, 11, 12]

# creating numpy array
sample_array = np.array([list_1,
                          list_2,
                          list_3])

print("Numpy array :")
print(sample_array)

# print shape of the array
print("Shape of the array :",
      sample_array.shape)
```

Output:

Numpy array:

```
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
```

Shape of the array: (3, 4)

Example:

```
import numpy as np

sample_array = np.array([[0, 4, 2],
                          [3, 4, 5],
                          [23, 4, 5],
                          [2, 34, 5],
                          [5, 6, 7]])

print("shape of the array :",
      sample_array.shape)
```

Output:

Shape of the array: (5, 3)

3. Rank: The rank of an array is simply the number of axes (or dimensions) it has.
The one-dimensional array has rank 1.

1	2	3	4	5
---	---	---	---	---

Rank 1

The two-dimensional array has rank 2.

1	2	3	4	5
6	7	8	9	10

Rank 2

4. Data type objects (dtype): Data type objects (dtype) is an instance of **numpy.dtype** class. It describes how the bytes in the fixed-size block of memory corresponding to an array item should be interpreted.

Example:

```
# Import module
import numpy as np

# Creating the array
sample_array_1 = np.array([[0, 4, 2]])

sample_array_2 = np.array([0.2, 0.4, 2.4])

# display data type
print("Data type of the array 1 :",
      sample_array_1.dtype)

print("Data type of array 2 :",
      sample_array_2.dtype)
```

Output:

Data type of the array 1 : int32

Data type of array 2 : float64

Some different way of creating Numpy Array :

1. numpy.array(): The Numpy array object in Numpy is called ndarray. We can create ndarray using **numpy.array()** function.

Syntax: *numpy.array(parameter)*

Example:

```
# import module
import numpy as np
#creating a array
arr = np.array([3,4,5,5])
print("Array :",arr)
```

Output:

Array: [3 4 5 5]

3. numpy.arange(): This is an inbuilt NumPy function that returns evenly spaced values within a given interval.

Syntax: *numpy.arange([start,]stop, [step,]dtype=None)*

Example:

```
import numpy as np

np.arange(1, 20 , 2,
         dtype = np.float32)
```

Output:

array([1., 3., 5., 7., 9., 11., 13., 15., 17., 19.], dtype=float32)

4. numpy.linspace(): This function returns evenly spaced numbers over a specified between two limits.

Syntax: *numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)*

Example 1:

```
import numpy as np

np.linspace(3.5, 10, 3)
```

Output:

array([3.5 , 6.75, 10.])

Example 2:

```
import numpy as np

np.linspace(3.5, 10, 3,
         dtype = np.int32)
```

Output:

array([3, 6, 10])

5. numpy.empty(): This function create a new array of given shape and type, without initializing value.

Syntax: *numpy.empty(shape, dtype=float, order='C')*

Example:

```
import numpy as np

np.empty([4, 3],
         dtype = np.int32,
         order = 'f')
```

Output:

```
array([[ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11],
       [ 4,  8, 12]])
```

7. **numpy.zeros()**: This function is used to get a new array of given shape and type, filled with zeros(0).

Syntax: *numpy.zeros(shape, dtype=None)*

Example:

```
import numpy as np
np.zeros([4, 3],
         dtype = np.int32,
         order = 'f')
```

Output:

```
array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

Accessing Array: by referring to its index number

Array referencing only the index of the required element has to be passed to the name of the array.

Syntax:

`array_name[index]`

- For referencing using numpy array, first an array is created using numpy's array function then it is referenced like a regular array.

Syntax:

`np.array([array elements])`

For 1 – dimension array

Creating a array of elements

```
arr = [4, 6, 3, 9, 2]
```

```

# Referring elements of the array
# by index to a new variable
first_element = arr[0]
second_element = arr[1]
third_element = arr[2]
fourth_element = arr[3]
fifth_element = arr[4]

# Print the variables
print("First Element =", first_element)
print("Second Element =", second_element)
print("Third Element =", third_element)
print("Fourth Element =", fourth_element)
print("Fifth Element =", fifth_element)

```

```

Creating a 2d-array of elements
arr = [[4, 6, 3],
       [5, 9, 2],
       [1, 8, 7]]

# Referring elements of the 2d-array
# by row and column index to new variables
first_row_second_column = arr[0][1]
second_row_first_column = arr[1][0]
third_row_third_column = arr[2][2]

# Print the variables
print("First Row Second Column =", first_row_second_column)
print("Second Row First Column =", second_row_first_column)
print("Third Row Third Column =", third_row_third_column)

```

Output:

```

Desktop/gfg')
First Row Second Column = 6
Second Row First Column = 5
Third Row Third Column = 7

```

Stacking & Splitting: stack(), array_split()

numpy.hstack() in Python

-

numpy.hstack() function is used to stack the sequence of input arrays horizontally (i.e. column wise) to make a single array.

Syntax : *numpy.hstack(tup)*

Parameters :

tup : [sequence of ndarrays] Tuple containing arrays to be stacked. The arrays must have the same shape along all but the second axis.

Return : [stacked ndarray] The stacked array of the input arrays.

Code #1 :

```
# Python program explaining
# hstack() function

import numpy as geek

# input array
in_arr1 = geek.array([ 1, 2, 3] )
print ("1st Input array : \n", in_arr1)

in_arr2 = geek.array([ 4, 5, 6] )
print ("2nd Input array : \n", in_arr2)

# Stacking the two arrays horizontally
out_arr = geek.hstack((in_arr1, in_arr2))
print ("Output horizontally stacked array:\n ", out_arr)
```

Output:

1st Input array:

[1 2 3]

2nd Input array:

[4 5 6]

Output horizontally stacked array:

[1 2 3 4 5 6]

Code #2:

```
# Python program explaining
# hstack() function

import numpy as geek

# input array
in_arr1 = geek.array([[ 1, 2, 3], [-1, -2, -3]] )
print ("1st Input array : \n", in_arr1)

in_arr2 = geek.array([[ 4, 5, 6], [-4, -5, -6]] )
print ("2nd Input array : \n", in_arr2)

# Stacking the two arrays horizontally
```

```
out_arr = geek.hstack((in_arr1, in_arr2))
print ("Output stacked array :\n ", out_arr)
```

Output:

1st Input array:

```
[[1 2 3]
```

```
[-1 -2 -3]]
```

2nd Input array:

```
[[4 5 6]
```

```
[-4 -5 -6]]
```

Output stacked array:

```
[[1 2 3 4 5 6]
```

```
[-1 -2 -3 -4 -5 -6]]
```

Splitting Arrays in Numpy

Array splitting can be vertical, horizontal, or depth-wise. We can use functions `hsplit()`, `vsplit()` and `dsplit()` respectively for the same. We can split arrays into arrays of the same shape by indicating the position after which the split should occur.

- **Horizontal splitting:** The '**hsplit()**' function splits an array along axis parameter = 1. '**numpy.hsplit**' is equivalent to '**split**' with **axis parameter = 1**, the array is always splitted along the second axis regardless of the array dimension. This function split an array into multiple sub-arrays horizontally (column-wise).

Syntax:

```
numpy.hsplit(ary, indices_or_sections)
```

Example:

```
# Horizontal array splitting using np.hsplit()
import numpy as np

# Making of a 3x3 array
a = np.arange(9).reshape(3, 3)
# Horizontal splitting of array
# 'a' using np.hsplit().
print("The array {} gets splitted \
horizontally to form {}".format(a, np.hsplit(a, 3)))
# Horizontal splitting of array 'a'
# using 'split' with axis parameter = 1.
print("The array {} gets splitted \
horizontally to form {}".format(a, np.split(a, 3, 1)))
```

Output:

The array [[0 1 2]

[3 4 5]

[6 7 8]] gets splitted horizontally to form [array([[0,

[3],

[6]]), array([[1],

[4],

[7]]), array([[2],

[5],

[8]])]

The array [[0 1 2]

[3 4 5]

[6 7 8]] gets splitted horizontally to form [array([[0,

[3],

[6]]), array([[1],

[4],

[7]]), array([[2],

[5],

[8]])]

- **Vertical splitting:** The '**vsplit()**' function splits an array along axis parameter = 0. '**numpy.vsplit**' is equivalent to '**split**' with **axis parameter = 0**. This function split an array into multiple sub-arrays vertically (row-wise).
`numpy.vsplit(ary, indices_or_sections)`

Example:

```
import numpy as np

# Making of a 3x3 array
a = np.arange(9).reshape(3, 3)

# Vertical splitting of array 'a'
# using np.vsplit().
print("The array {} gets splitted \
vertically to form {}".format(a, np.vsplit(a, 3)))

# Vertical splitting of array 'a'
# using 'split' with axis parameter = 0.
```

```
print("The array {} gets splitted \
vertically to form {}".format(a, np.split(a, 3, 0)))
```

Output:

The array [[0 1 2]

[3 4 5]

[6 7 8]] gets splitted vertically to form [array([[0, 1, 2]]), array([[3, 4, 5]]), array([[6, 7,

8]])]

The array [[0 1 2]

[3 4 5]

[6 7 8]] gets splitted vertically to form [array([[0, 1, 2]]), array([[3, 4, 5]]), array([[6, 7,

8]])]

- **Depth-wise splitting:** It Split the array into multiple sub-arrays along the 3rd axis (depth).

`numpy.dsplit(ary, indices_or_sections)`

Example:

```
import numpy as np

# Making of a 3x3x3 array.
b = np.arange(27).reshape(3, 3, 3)

# Depth-wise splitting of array
# 'b' using np.dsplit().
print("The array {} gets splitted \
depth-wise to form {}".format(b, np.dsplit(b, 3)))
```

Output:

The array [[[0 1 2]

[3 4 5]

[6 7 8]]

[[9 10 11]

[12 13 14]

[15 16 17]]

[[18 19 20]

[21 22 23]

[24 25 26]]] gets splitted depth-wise to form [array([[[0],

[3],

[6]],

[[9],

[12],

[15]],

```

[[18],
 [21],
 [24]]], array([[[ 1],
 [ 4],
 [ 7]],

 [[10],
 [13],
 [16]],

 [[19],
 [22],
 [25]]]), array([[[ 2],
 [ 5],
 [ 8]],

 [[11],
 [14],
 [17]],

 [[20],
 [23],
 [26]]])

```

Maths function of Numpy

numpy.add() in Python

numpy.add() function is used when we want to compute the addition of two array. It add arguments element-wise. If shape of two arrays are not same, that is `arr1.shape != arr2.shape`, they must be broadcastable to a common shape (which may be the shape of one or the other).

```

import numpy as geek
in_num1 = 10
in_num2 = 15

print ("1st Input number :", in_num1)
print ("2nd Input number :", in_num2)

out_num = geek.add(in_num1, in_num2)
print ("output number after addition :", out_num)

```

Output :

```

1st Input number : 10
2nd Input number : 15
output number after addition : 25

```

numpy.subtract() in Python

numpy.subtract() function is used when we want to compute the difference of two array. It returns the difference of arr1 and arr2, element-wise.

```
import numpy as geek
in_num1 = 4
in_num2 = 6

print ("1st Input number : ", in_num1)
print ("2nd Input number : ", in_num2)

out_num = geek.subtract(in_num1, in_num2)
print ("Difference of two input number : ", out_num)
```

Output :

1st Input number : 4

2nd Input number : 6

Difference of two input number : -2

**Python program explaining
numpy.subtract() function**

```
import numpy as geek

in_arr1 = geek.array([[2, -4, 5], [-6, 2, 0]])
in_arr2 = geek.array([[0, -7, 5], [5, -2, 9]])

print ("1st Input array : ", in_arr1)
print ("2nd Input array : ", in_arr2)

out_arr = geek.subtract(in_arr1, in_arr2)
print ("Output array: ", out_arr)
```

Output :

1st Input array : [[2 -4 5]

[-6 2 0]]

2nd Input array : [[0 -7 5]

[5 -2 9]]

Output array: [[2 3 0]

[-11 4 -9]]

numpy.multiply() in Python

numpy.multiply() function is used when we want to compute the multiplication of two array. It returns the product of arr1 and arr2, element-wise.

Example #1 :

```
# Python program explaining
# numpy.multiply() function

import numpy as geek
in_num1 = 4
in_num2 = 6

print ("1st Input number : ", in_num1)
print ("2nd Input number : ", in_num2)

out_num = geek.multiply(in_num1, in_num2)
print ("output number : ", out_num)
```

Output :

1st Input number : 4

2nd Input number : 6

output number : 24

Example #2 :

```
# Python program explaining
# numpy.multiply() function

import numpy as geek

in_arr1 = geek.array([[2, -7, 5], [-6, 2, 0]])
in_arr2 = geek.array([[0, -7, 8], [5, -2, 9]])

print ("1st Input array : ", in_arr1)
print ("2nd Input array : ", in_arr2)
out_arr = geek.multiply(in_arr1, in_arr2)
print ("Resultant output array: ", out_arr)
```

Output :

1st Input array : [[2 -7 5]

[-6 2 0]]

2nd Input array : [[0 -7 8]

[5 -2 9]]

Resultant output array: [[0 49 40]

[-30 -4 0]]

numpy.divide() in Python

numpy.divide(arr1, arr2, out = None, where = True, casting = 'same_kind', order = 'K', dtype = None) :

Code 1: arr1 divide by arr2 elements

```
# Python program explaining
# divide() function
import numpy as np

# input_array
arr1 = [2, 27, 2, 21, 23]
arr2 = [2, 3, 4, 5, 6]
print ("arr1      : ", arr1)
print ("arr2      : ", arr2)

# output_array
out = np.divide(arr1, arr2)
print ("\nOutput array : \n", out)
```

Output:

arr1 : [2, 27, 2, 21, 23]

arr2 : [2, 3, 4, 5, 6]

Output array:

[1. 9. 0.5 4.2 3.83333333]

Code 2: elements of arr1 divided by divisor

```
# Python program explaining
# divide() function
import numpy as np

# input_array
arr1 = [2, 27, 2, 21, 23]
divisor = 3
print ("arr1      : ", arr1)

# output_array
out = np.divide(arr1, divisor)
print ("\nOutput array : \n", out)
```

Output:

arr1 : [2, 27, 2, 21, 23]

Output array:

```
[ 0.66666667  9.      0.66666667  7.      7.66666667]
```

numpy.power() in Python

-

numpy.power(arr1, arr2, out = None, where = True, casting = 'same_kind', order = 'K', dtype = None) :

Array element from first array is raised to the power of element from second element(all happens element-wise). Both arr1 and arr2 must have same shape and each element in arr1 must be raised to corresponding +ve value from arr2; otherwise it will raise a **ValueError**

Code 1 : arr1 raised to arr2

```
# Python program explaining
# power() function
import numpy as np

# input_array
arr1 = [2, 2, 2, 2, 2]
arr2 = [2, 3, 4, 5, 6]
print ("arr1      : ", arr1)
print ("arr2      : ", arr2)

# output_array
out = np.power(arr1, arr2)
print ("\nOutput array : ", out)
```

Output :

```
arr1      : [2, 2, 2, 2, 2]
```

```
arr2      : [2, 3, 4, 5, 6]
```

Output array : [4 8 16 32 64]

Code 2 : elements of arr1 raised to exponent 2

```
# Python program explaining
# power() function
import numpy as np

# input_array
arr1 = np.arange(8)
exponent = 2
print ("arr1      : ", arr1)
```

```
# output_array
out = np.power(arr1, exponent)
print ("\nOutput array : ", out)
```

Output :

arr1 : [0 1 2 3 4 5 6 7]

Output array : [0 1 4 9 16 25 36 49]

numpy.mod() function in Python is used for performing mathematical operations on numpy arrays. It returns the element-wise remainder of division between two arrays

```
import numpy as geek
in_arr1 = geek.array([2, -4, 7])
in_arr2 = geek.array([2, 3, 4])
print("Dividend array: ", in_arr1)
print("Divisor array: ", in_arr2)
out_arr = geek.mod(in_arr1, in_arr2)
print("Output remainder array: ", out_arr)
```

This will output:

Dividend array: [2 -4 7]

Divisor array: [2 3 4]

Output remainder array: [0 2 3]

Statistics Functions: **amin()**, **amax()**, **mean()**, **median()**, **std()**, **var()**, **average()**, **ptp()**

The **numpy.amin()** function returns minimum of an array or minimum along axis(if mentioned). **Syntax** : `numpy.amin(arr, axis = None, out = None, keepdims = <class numpy._globals._NoValue>)`

```
import numpy as geek
```

```
# Creating a 1-D array
```

```
arr = geek.arange(8)
print("arr : ", arr)
print("Min of arr : ", geek.amin(arr))
```

```
# Creating a 2-D array
```

```
arr = geek.arange(10).reshape(2, 5)
print("\narr : ", arr)
print("\nMin of arr, axis = None : ", geek.amin(arr))
print("Min of arr, axis = 0 : ", geek.amin(arr, axis = 0)) # Minimum along the first axis (vertical)
```

```
print("Min of arr, axis = 1 : ", geek.amin(arr, axis = 1)) # Minimum along the second axis (horizontal)
```

This will output:

```
arr : [0 1 2 3 4 5 6 7]
```

```
Min of arr : 0
```

```
arr : [[0 1 2 3 4] [5 6 7 8 9]]
```

```
Min of arr, axis = None : 0
```

```
Min of arr, axis = 0 : [0 1 2 3 4]
```

```
Min of arr, axis = 1 : [0 5]
```

The numpy.amax() method returns the maximum of an array or maximum along the axis(if mentioned). **Syntax:**

```
numpy.amax(arr, axis = None, out = None, keepdims = <class numpy._globals._NoValue>)
```

```
import numpy as geek
```

```
# 1D array
```

```
arr = geek.arange(8)
```

```
print("Max of arr: ", geek.amax(arr))
```

```
# 2D array
```

```
arr = geek.arange(10).reshape(2, 5)
```

```
print("Max of arr, axis = None: ", geek.amax(arr)) # Maximum of the flattened array
```

```
print("Max of arr, axis = 0: ", geek.amax(arr, axis = 0)) # Maxima along the first axis (vertical)
```

```
print("Max of arr, axis = 1: ", geek.amax(arr, axis = 1)) # Maxima along the second axis (horizontal)
```

This will output:

```
Max of arr: 7
```

```
Max of arr, axis = None: 9
```

```
Max of arr, axis = 0: [5 6 7 8 9]
```

```
Max of arr, axis = 1: [4 9]
```

- **mean:** This function returns the arithmetic mean of array elements. The mean is the sum of all array elements divided by the number of elements. Here's how you can use it:

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
print(np.mean(a))
```

- **median:** This function returns the median of the array elements. The median is the value separating the higher half from the lower half of a data sample. Here's how you can use it:

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
print(np.median(a))
```

- **std**: This function returns the standard deviation of the array elements. The standard deviation is a measure of the amount of variation or dispersion of a set of values. Here's how you can use it:

```
import numpy as np
a = np.array([1, 2, 3])
print(np.std(a))
```
- **var**: This function returns the variance of the array elements. The variance is a measure of dispersion, being equal to the average squared deviation from the mean. Here's how you can use it:

```
import numpy as np
a = np.array([1, 2, 3])
print(np.var(a))
```
- **average**: This function computes the weighted average along the specified axis. Here's how you can use it:

```
import numpy as np
a = np.array([1, 2, 3])
weights = np.array([0.3, 0.3, 0.4])
print(np.average(a, weights=weights))
```
- **ptp (Peak to Peak)**: This function returns the range of values (maximum - minimum) along an axis. Here's how you can use it:

```
import numpy as np
a = np.array([1, 2, 3])
print(np.ptp(a))
```

PANDAS LIBRARY

A Pandas Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.). The axis labels are collectively called the index. A Pandas Series is nothing but a column in an excel sheet¹. Here are some basic operations which can be performed on Pandas Series¹:

- **Creating a Series**: You can create a Pandas Series by loading the datasets from existing storage, storage can be SQL Database, CSV file, and Excel file. Pandas Series can also be created from the lists, dictionary, and from a scalar value etc¹

.Accessing element of Series: There are two ways through which we can access Accessing Element Using Label (index)¹.

- **Indexing and Selecting Data in Series**
- **Binary operation on Series**
- **Conversion Operation on Series**

Here is an example of creating a series from an array:

```
# import pandas as pd
import pandas as pd

# import numpy as np
import numpy as np

# simple array
data = np.array(['g','e','e','k','s'])

ser = pd.Series(data)
print(ser)
```

A DataFrame in pandas is a two-dimensional, size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). It consists of three principal components: the data, rows, and columns¹.

1. **Creating a DataFrame using a List¹:**

```
import pandas as pd
lst = ['Geeks', 'For', 'Geeks', 'is', 'portal', 'for', 'Geeks']
df = pd.DataFrame(lst)
print(df)
```

2. **Creating DataFrame from dict of ndarray/lists¹:**

```
import pandas as pd
data = {'Name': ['Tom', 'nick', 'krish', 'jack'], 'Age': [20, 21, 19, 18]}
df = pd.DataFrame(data)
print(df)
```

3. **Column Selection¹:**

Python This code is AI-generated. Review and use carefully. [Visit our FAQ](#) for more information.

```
import pandas as pd
data = {'Name': ['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age': [27, 24, 22, 32],
        'Address': ['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'],
        'Qualification': ['Msc', 'MA', 'MCA', 'Phd']}
df = pd.DataFrame(data)
print(df[['Name', 'Qualification']])
```

4. **Row Selection²:**

Python This code is AI-generated. Review and use carefully. [Visit our FAQ](#) for more information.

```
import pandas as pd
data = {"calories": [420, 380, 390], "duration": [50, 40, 45]}
df = pd.DataFrame(data)
print(df.loc[0])
```

In the real world, a Data Frame will often be created by loading datasets from existing storage like SQL Databases, CSV files, and Excel files¹. However, you can also create Data Frames from lists, dictionaries, and from a list of dictionaries

The read_csv() function in pandas is used to read a CSV file into DataFrame. Here is a basic example of how to use it:

```
import pandas as pd
```

```
# Specify the file path or URL
file_path = 'your_file_path.csv'
```

```
# Read the CSV file
df = pd.read_csv(file_path)
```

```
# Display the DataFrame
print(df)
```

In this code:

- `pd.read_csv(file_path)` reads the CSV file specified by `file_path` into a DataFrame `df`.
- `print(df)` displays the content of the DataFrame.

The `read_csv()` function has many parameters that you can use to customize the import process, such as specifying a delimiter with `sep`, handling missing values, setting data types for columns, and more. You can find more information in the pandas documentation.

1 of 10

Cleaning Empty Cells: dropna()

- **dropna()**: This function is used to remove missing values. You can specify parameters like `axis` (to drop rows or columns), `how` (to drop if any or all values are null), and `thresh` (to require a certain number of non-NA values).

Cleaning Wrong Data: drop()

- **drop()**: This function is used to drop specified labels from rows or columns. You can specify parameters like `labels` (Index or column labels to drop) and `axis` (Whether to drop labels from the index (0 or 'index') or columns (1 or 'columns')).

Removing Duplicates: duplicated()

- **duplicated()**: This function returns a boolean Series denoting duplicate rows. You can specify parameters like subset (Only consider certain columns for identifying duplicates) and keep (To control how to consider duplicate value).

Pandas Plotting: plot()

- **plot()**: This function makes plots of Series or DataFrame. It is a wrapper around matplotlib.pyplot.plot(), and the result is a matplotlib subplot object.

```
import pandas as pd

# Create a simple dataframe
df = pd.DataFrame({
    'A': ['foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'foo'],
    'B': ['one', 'one', 'two', 'three', 'two', 'two', 'one', 'three'],
    'C': np.random.randn(8),
    'D': np.random.randn(8)
})

# Drop duplicates
df = df.drop_duplicates()

# Drop any rows with missing data
df = df.dropna()

# Drop a specific column (in this case, column 'D')
df = df.drop(columns=['D'])

# Plot the data in column 'C'
df['C'].plot()
```

Matplotlib Library

It seems like you're referring to the plot() and show() functions from the matplotlib.pyplot module in Python.

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 3, 5, 7, 11]
```

```
# Create a figure and a set of subplots
```

```
plt.figure()
```

```
# Plot y versus x as lines and/or markers
```

```
plt.plot(x, y)
```

```
# Display all open figures
plt.show()
```

In this code:

- `import matplotlib.pyplot as plt` imports the pyplot module from matplotlib and gives it the alias `plt`.
 - `x = [1, 2, 3, 4, 5]` and `y = [2, 3, 5, 7, 11]` are lists of sample data.
 - `plt.figure()` creates a new figure.
 - `plt.plot(x, y)` plots `y` versus `x` as lines and/or markers.
 - `plt.show()` displays all open figures. This will open a window with the plot.
-
- `xlabel('string')` and `ylabel('string')`: These functions are used to set the labels for the `x` and `y` axes respectively. Replace 'string' with the label you want to set.
 - `grid(True)`: This function is used to add grid lines to your plot. The argument `True` turns the grid lines on.
 - `bar(x, height)`: This function is used to make a bar plot. `x` is the `x` coordinates of the bars, and `height` is the heights of the bars.
 - `hist(x)`: This function is used to plot a histogram. `x` is a list or array of data values.
 - `subplot(nrows, ncols, index)`: This function is used to add a subplot to the current figure. `nrows` and `ncols` are used to construct a grid of subplots, and `index` is used to identify the particular subplot that this function is to create within the notional grid.
 - `pie(x)`: This function is used to plot a pie chart. `x` is a list or array of data values.

```
import matplotlib.pyplot as plt
```

```
# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
```

```
# Create a figure and a set of subplots
plt.figure()
```

```
# Plot y versus x as lines and/or markers
plt.plot(x, y)
```

```
# Set labels for x and y axes
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
# Add grid lines
```

```
plt.grid(True)
```

```
# Create a bar plot in a new subplot
```

```
plt.subplot(2, 1, 1)
```

```
plt.bar(x, y)
```

```
# Create a histogram in a new subplot
```

```
plt.subplot(2, 1, 2)
```

```
plt.hist(y)
```

```
# Display all open figures
```

```
plt.show()
```

Save the plotted images into pdf: savefig()

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Create some data
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
# Create a figure and plot the data
```

```
plt.figure()
```

```
plt.plot(x, y)
```

```
# Save the figure to a PDF file
```

```
plt.savefig('output.pdf', format='pdf')
```

In this code:

- We first import the necessary libraries.
- We then create some data to plot.
- We create a figure and plot the data.
- Finally, we save the figure to a PDF file named 'output.pdf' using the savefig() function. The format parameter is used to specify the format of the output file.

Please replace 'output.pdf' with your desired output file path and name. The file will be saved in the same directory as your Python script if no path is specified. If the directory does not exist, you will get an error. So, make sure that the directory where you want to save the file exists. Also, don't forget to close the plot using plt.close() if you are creating multiple plots in a loop. This is to free up memory

Scikit-learn (sklearn) is a powerful Python library for machine learning. It contains a lot of efficient tools for machine learning and statistical modeling including classification,

regression, clustering and dimensionality reduction via a consistent interface in Python. Here are the key concepts and steps to build a model in sklearn:

Key Concepts:

1. **Estimators:** Any object that can estimate some parameters based on a dataset is called an estimator. The estimation itself is performed by the `fit()` method.
2. **Transformers:** Some estimators can also transform a dataset; these are called transformers. The transformation is performed by the `transform()` method.
3. **Predictors:** Some estimators are capable of making predictions given a dataset; they are called predictors. A predictor has a `predict()` method.

Steps to Build a Model:

1. **Loading a Dataset:** You can use pandas' `read_csv()` function to load your dataset into a DataFrame.

```
import pandas as pd
data = pd.read_csv('your_data.csv')
```

2. **Preprocessing:** Depending on your dataset, you might need to preprocess your data, like filling missing values, converting categorical data to numerical data etc.
3. **Splitting Data:** You can use sklearn's `train_test_split()` function to split your data into training and testing sets.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

4. **Creating a Model:** Choose an appropriate estimator for your problem (like `LinearRegression` for a regression problem).

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

5. **Training the Model:** You can train/fit your model on your training data using the `fit()` method.

```
model.fit(X_train, y_train)
```

6. **Making Predictions:** Once the model is trained, you can make predictions on your test data using the `predict()` method.

```
predictions = model.predict(X_test)
```

7. **Evaluating the Model:** You can use different metrics available in sklearn like `mean_squared_error` for regression problems and `accuracy_score` for classification problems to evaluate how well your model performed.

UNIT 3: Preparing to Model and Preprocessing

Machine Learning activities

Preparing to Model

The process of preparing a model in machine learning involves several steps¹²³:

1. Collecting Data: Gather data from relevant sources. The quality of the data you feed to the machine will determine how accurate your model is³.
2. Preparing the Data: This involves cleaning, analyzing, and visualizing the data¹. Check for any missing, duplicate or inconsistent data and clean it. This may include removing any irrelevant columns, filling in missing values, and formatting data correctly².
3. Choosing a Model: Select a suitable model based on the problem and the data¹.
4. Training the Model: Pass the prepared data to your machine learning model to find patterns and make predictions³.
5. Evaluating the Model: Use appropriate metrics and methods to assess the performance of the model¹.
6. Tuning the Model: Optimize the results by adjusting the model parameters¹.
7. Using the Model for Prediction: Apply the model to make predictions and present the outcome

Learning: Data Partition-k-fold cross validation

K-fold cross-validation is a technique used in machine learning to evaluate the performance of a model on unseen data¹. Here's how it works:

1. The available data is divided into k subsets, or folds¹.
2. For each unique group, we take a turn to hold out one fold as the test set¹.
3. The remaining k-1 folds are used as the training set¹.
4. We train our model on the training set and test it on the test set¹.
5. Repeat this process k times, each time with a different fold as our test set¹.
6. Finally, we average the results to get our final model¹.

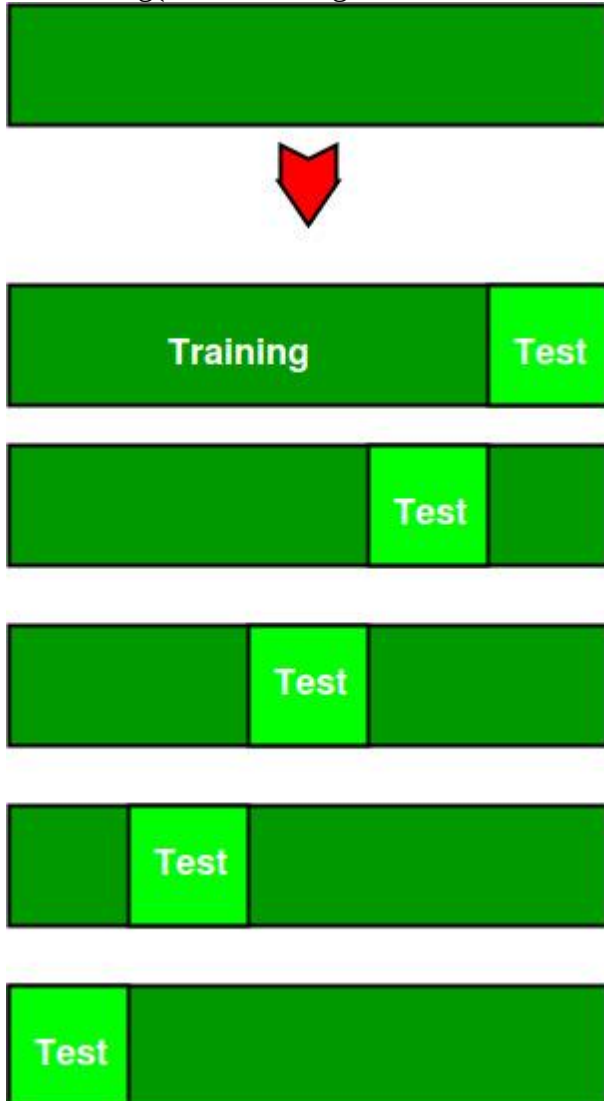
The main purpose of k-fold cross-validation is to prevent overfitting, which occurs when a model is trained too well on the training data and performs poorly on new, unseen data¹. By evaluating the model on multiple validation sets, k-fold cross-validation provides a more realistic estimate of the model's generalization performance, i.e., its ability to perform well on new, unseen data¹.

There are several types of cross-validation techniques, including leave-one-out cross-validation, and stratified cross-validation¹. The choice of technique depends on the size and nature of the data, as well as the specific requirements of the modeling problem¹.

In summary, k-fold cross-validation is an important step in the machine learning process and helps to ensure that the model selected for deployment is robust and generalizes well to new data

Example The diagram below shows an example of the training subsets and evaluation subsets generated in k-fold cross-validation. Here, we have total 25 instances. In first iteration we use the first 20 percent of data for evaluation, and the remaining 80 percent for training([1-5] testing and [5-25] training) while in the second iteration we use the second subset of 20 percent for evaluation, and the remaining three subsets of the data

for training([5-10] testing and [1-5 and 10-25] training), and so on.



Total instances: 25

1. Value of k : 5

No. Iteration	Training set observations	Testing set
1	[5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]	[0 1 2 3 4]
2	[0 1 2 3 4 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]	[5 6 7 8 9]
3	[0 1 2 3 4 5 6 7 8 9 15 16 17 18 19 20 21 22 23 24]	[10 11 12 13 14]
4	[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 20 21 22 23 24]	[15 16 17 18 19]
5	[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19]	[20 21 22 23 24]

Model Selection

Performance Evaluation:

Performance evaluation in **Machine Learning (ML)** is the process of measuring how well a model performs on unseen data.

Why Performance Evaluation is Important

- Measures the accuracy and reliability of a model.
- Helps compare different models.
- Detects problems such as overfitting and underfitting.
- Ensures the model generalizes well to new data.

Confusion Matrix

A confusion matrix is a matrix that summarizes the performance of a machine learning model on a set of test data. It is often used to measure the performance of **classification** models, which aim to predict a categorical label for each input instance. A confusion matrix summarizes prediction results. The matrix displays the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) produced by the model on the test data.

For binary classification, the matrix will be of a 2X2 table, For multi-class classification, the matrix shape will be equal to the number of classes i.e for n classes it will be nXn.

Actual / Predicted		Positive	Negative
Positive		True Positive (TP)	False Negative (FN)
Negative		False Positive (FP)	True Negative (TN)

A 2X2 Confusion matrix is shown below for the image recognition having a Dog image or Not Dog image.

		Actual	
		Dog	Not Dog
Predicted	Dog	True Positive (TP)	False Positive (FP)

		Actual	
		Dog	Not Dog
	Not Dog	False Negative (FN)	True Negative (TN)

- True Positive (TP): It is the total counts having both predicted and actual values are Dog.
- True Negative (TN): It is the total counts having both predicted and actual values are Not Dog.
- False Positive (FP): It is the total counts having prediction is Dog while actually Not Dog.
- False Negative (FN): It is the total counts having prediction is Not Dog while actually, it is Dog.

Example

Index	1	2	3	4	5	6	7	8	9	10
Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5

- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

		Actual	
		Dog	Not Dog
Predicted	Dog	True Positive (TP =5)	False Positive (FP=1)
	Not Dog	False Negative (FN =1)	True Negative (TN=3)

Performance Improvement: Ensemble Learning

Ensemble Learning is a machine learning technique that combines multiple models (called **base learners** or **weak learners**) to produce a more accurate and robust prediction than a single model.

Definition

Ensemble learning improves model performance by combining the predictions of multiple models. The main idea is that a group of models can make better decisions than an individual model.

Why Use Ensemble Learning?

- Improves prediction accuracy.
- Reduces overfitting.
- Increases model stability and robustness.
- Handles complex datasets more effectively.

Types of Data

Qualitative/Categorical Data: Nominal, Ordinal

Quantitative/Numeric Data: Interval, Ratio

In machine learning, data is often categorized as:

1. **Qualitative/Categorical Data**: This type of data is used to categorize objects or to describe them in a qualitative manner. It's further divided into:
 - Nominal: This data type represents discrete units and is used to label variables without any quantitative value. For example, gender (Male/Female) or car color (Red/Blue/Green).
 - Ordinal: This data type represents discrete and ordered units. It's similar to nominal data but it has a meaningful order or position. For example, ratings given to a restaurant by customers (Happy, Neutral, Unhappy).
 -
2. **Quantitative/Numeric Data**: This type of data is numerical, measurable quantities in which arithmetic operations often make sense. It's further divided into:
 - Interval: This data type represents values that can be ordered and have a meaningful difference. The difference between two values is meaningful. For example, temperature in Celsius (because the difference between 20°C and 30°C is the same as between 30°C and 40°C).
 - Ratio: This data type is similar to interval data, but it has a clear definition of zero. For example, height or weight can be ratio variables, as they can be measured from zero.

These different types of data require different statistical and machine learning methods and different visualization techniques.

Data quality and remediation

Handling outliers

Handling missing values

Data Quality and Remediation Data quality refers to the state of qualitative or quantitative pieces of information. Good data quality exists when data is accurate, complete, consistent, reliable, and up-to-date¹. Data remediation is the process of cleaning, organizing, and transforming data to ensure its quality². It involves correcting errors and mistakes in data to eliminate data-quality issues². This is crucial because poor data quality can lead to inaccurate models and eventually poor performance¹.

Handling Outliers Outliers are data points that deviate significantly from other observations. They can be caused by measurement uncertainty or experimental error³. Outliers can be detected using various techniques such as visualization (e.g., box plots) or statistical methods³. Once detected, outliers can be handled in several ways:

- Removing all the outliers⁴.
- Replacing outlier values with a suitable value⁴.
- Using algorithms that are less sensitive to outliers⁵.

Handling Missing Values in Machine Learning Missing values in a dataset can occur for various reasons such as errors during data collection or non-responses⁶. Handling missing values is important as they can bias the results of machine learning models and/or reduce their accuracy⁶. Some common strategies for handling missing values include:

- Deleting the missing values⁷.
- Imputing the missing values with statistical measures like mean, median, mode, etc⁷.
- Using algorithms that can handle missing values⁵.
- The choice of method for handling outliers and missing values can depend on factors like the nature of the data, the proportion of outliers or missing values, and the specific requirements of the analysis or model. It's always a good idea to explore different approaches and understand their implications.

Data Pre-Processing

Dimensionality reduction

Feature subset selection: Filter, Wrapper, Hybrid, and Embedded

Data pre-processing is a crucial step in the data mining process. It transforms raw data into an understandable format and involves cleaning, normalization, transformation, feature extraction, and selection. Here's a brief explanation of the methods you mentioned:

- **Dimensionality Reduction:** This technique is used to reduce the number of input variables in a dataset. More input features often make a predictive modeling task more challenging to model, more generally referred to as the curse of dimensionality.
- **Feature Subset Selection:** This is the process of selecting a subset of relevant features for use in model construction. The central assumption when using a feature selection technique is that the data contains many redundant or irrelevant features. Feature selection techniques are often used in domains where there are many features and comparatively few samples (or data points).
- There are four main types of feature subset selection methods:
 1. **Filter Methods:** These methods apply a statistical measure to assign a scoring to each feature. The features are ranked by the score and either selected to be kept or removed from the dataset. Examples include Chi-Squared Test, information gain, and correlation coefficient scores.
 2. **Wrapper Methods:** These methods consider the selection of a set of features as a search problem, where different combinations are prepared, evaluated and compared to other combinations. A predictive model is used to evaluate a combination of features and assign a score based on model accuracy. Examples include recursive feature elimination algorithm.
 3. **Hybrid Methods:** These methods are a combination of filter and wrapper methods by using a filter method to reduce search space that will be considered by the subsequent wrapper.
 4. **Embedded Methods:** These methods learn which features best contribute to the accuracy of the model while the model is being created. The most common type of embedded feature selection methods are regularization methods.

UNIT 4: Supervised Machine Learning Models

Brief explanation of Supervised Machine Learning

Supervised Machine Learning is a type of machine learning where the model is trained on a labeled dataset. Here's a brief explanation:

In Supervised Learning, we have a dataset consisting of both features and labels. The task is to construct an estimator which is able to predict the label of an object given the set of features. A wide range of supervised learning algorithms are available, each with its strengths

and weaknesses. There is no single learning algorithm that works best on all supervised learning problems.

The process involves four steps:

1. **Data Acquisition:** This involves collecting or creating a dataset with features and corresponding labels.
2. **Model Training:** The machine learning algorithm (like linear regression, decision tree, etc.) uses this labeled dataset to understand the relationship between features and labels.
3. **Model Evaluation:** The model's performance is evaluated, typically using a separate testing dataset. Common metrics include accuracy, precision, recall, F1 score, etc.
4. **Prediction:** Once the model is trained and evaluated, it can be used to predict labels for new, unseen data.

A classic example of supervised learning is email spam detection. The machine learning model is trained on emails that are labeled as "spam" or "not spam". It learns from these labels to predict whether a new email is spam or not.

Working of Supervised Machine learning

Supervised learning is the types of machine learning in which machines are trained using well "labelled" training data, and on basis of that data, machines predict the output. The labelled data means some input data is already tagged with the correct output.

In supervised learning, the training data provided to the machines work as the supervisor that teaches the machines to predict the output correctly. It applies the same concept as a student learns in the supervision of the teacher.

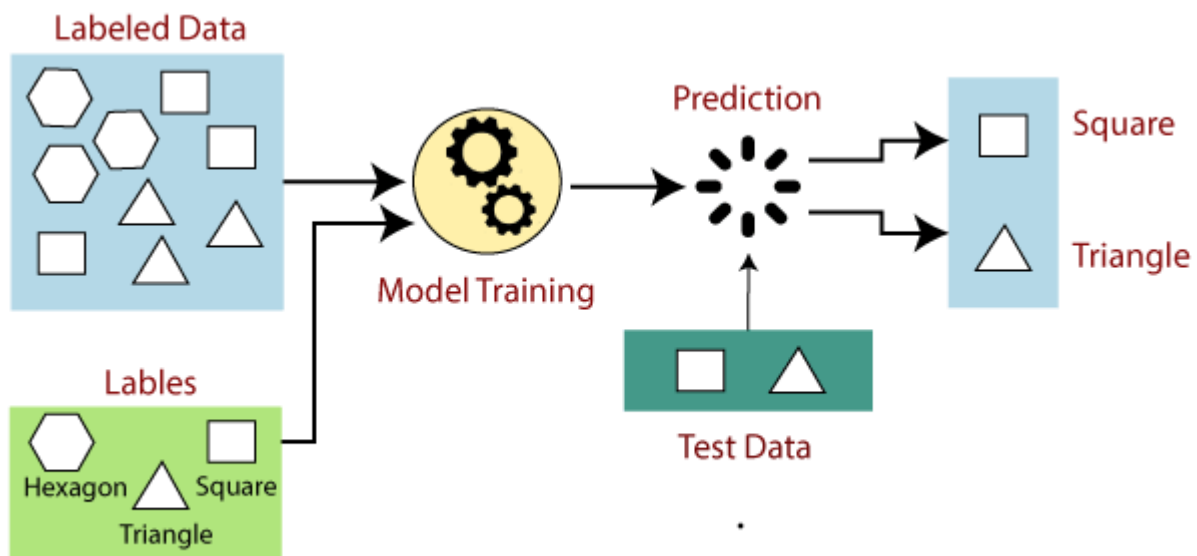
Supervised learning is a process of providing input data as well as correct output data to the machine learning model. The aim of a supervised learning algorithm is to **find a mapping function to map the input variable(x) with the output variable(y)**.

In the real-world, supervised learning can be used for **Risk Assessment, Image classification, Fraud Detection, spam filtering**, etc.

How Supervised Learning Works?

In supervised learning, models are trained using labelled dataset, where the model learns about each type of data. Once the training process is completed, the model is tested on the basis of test data (a subset of the training set), and then it predicts the output.

The working of Supervised learning can be easily understood by the below example and diagram:



Suppose we have a dataset of different types of shapes which includes square, rectangle, triangle, and Polygon. Now the first step is that we need to train the model for each shape.

- If the given shape has four sides, and all the sides are equal, then it will be labelled as a **Square**.
- If the given shape has three sides, then it will be labelled as a **triangle**.
- If the given shape has six equal sides then it will be labelled as **hexagon**.

Now, after training, we test our model using the test set, and the task of the model is to identify the shape.

The machine is already trained on all types of shapes, and when it finds a new shape, it classifies the shape on the bases of a number of sides, and predicts the output.

Steps Involved in Supervised Learning:

- First Determine the type of training dataset
- Collect/Gather the labelled training data.
- Split the training dataset into training **dataset**, **test dataset**, and **validation dataset**.
- Determine the input features of the training dataset, which should have enough knowledge so that the model can accurately predict the output.
- Determine the suitable algorithm for the model, such as support vector machine, decision tree, etc.
- Execute the algorithm on the training dataset. Sometimes we need validation sets as the control parameters, which are the subset of training datasets.

- Evaluate the accuracy of the model by providing the test set. If the model predicts the correct output, which means our model is accurate.

Steps involved in Supervised Machine Learning:

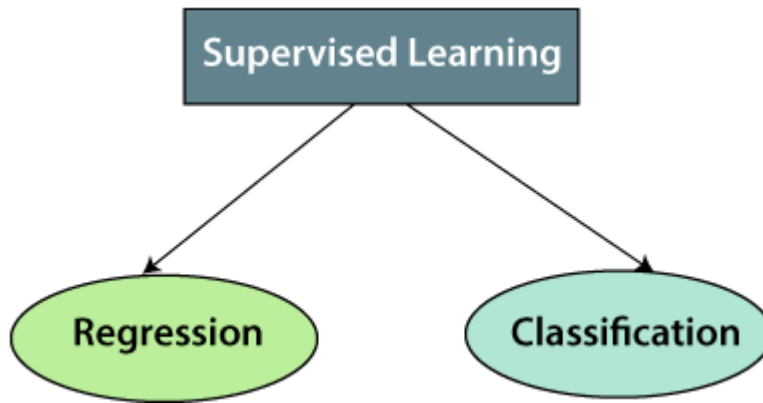
1. **Problem Definition:** Define the problem and determine the type of supervised learning to apply. This could be classification (predicting discrete values) or regression (predicting continuous values).
2. **Data Collection:** Gather the labelled training data. The quality and quantity of the data collected will directly determine how good the predictive model can be.
3. **Data Preparation:** Preprocess the data to make it suitable for use in machine learning algorithms. This could involve data cleaning, transformation, And reduction.
4. **Feature Selection:** Identify the input variables (or features) that will be used to create the predictive model.
5. **Algorithm Selection:** Choose an appropriate machine learning algorithm based on the problem type, data type, and business requirements.
6. **Model Training:** Train the model using the selected algorithm and training data. The model learns from this data.
7. **Model Evaluation:** Test the model's performance using a separate dataset (test set). Evaluate the accuracy of the model.
8. **Parameter Tuning:** Based on the performance on the test set, adjust the parameters of the algorithm to improve its accuracy.
9. **Prediction:** Use the trained model to make predictions on new unseen data

Model Deployment: Once satisfied with its performance, deploy the model in a suitable system where it can take in new data and predict outcomes in real-time.

2of10

Types of supervised Machine learning Algorithms:

Supervised learning can be further divided into two types of problems:



1. Regression

Regression algorithms are used if there is a relationship between the input variable and the output variable. It is used for the prediction of continuous variables, such as Weather forecasting, Market Trends, etc. Below are some popular Regression algorithms which come under supervised learning:

- Linear Regression
- Regression Trees
- Non-Linear Regression
- Bayesian Linear Regression
- Polynomial Regression

2. Classification

Classification algorithms are used when the output variable is categorical, which means there are two classes such as Yes-No, Male-Female, True-false, etc.

Spam Filtering,

- Random Forest
- Decision Trees
- Logistic Regression
- Support vector Machines

Types of ML Classification Algorithms:

Classification Algorithms can be further divided into the Mainly two category:

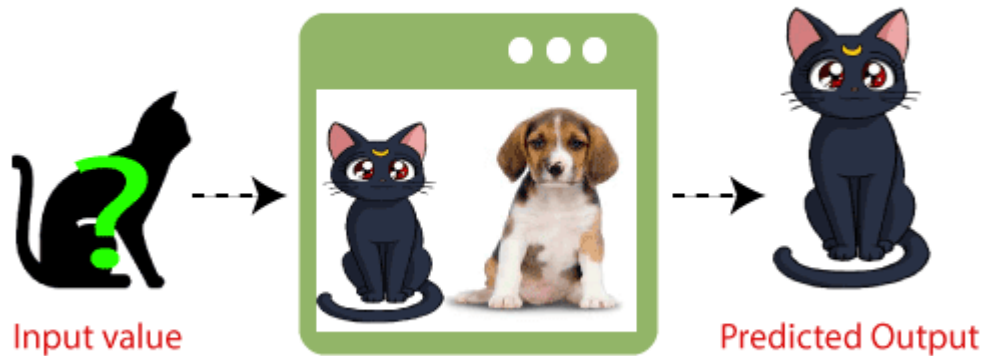
- **Linear Models**
 - Logistic Regression
 - Support Vector Machines

- **Non-linear Models**
 - K-Nearest Neighbours
 - Kernel SVM
 - Naïve Bayes
 - Decision Tree Classification
 - Random Forest Classification

K-Nearest Neighbor(KNN) Algorithm for Machine Learning

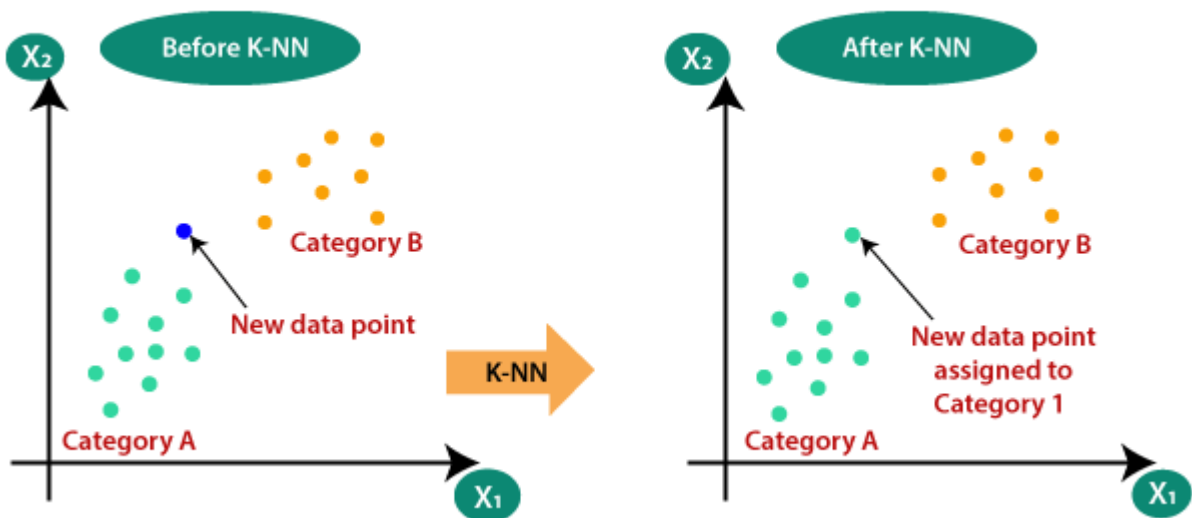
- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

KNN Classifier



Why do we need a K-NN Algorithm?

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



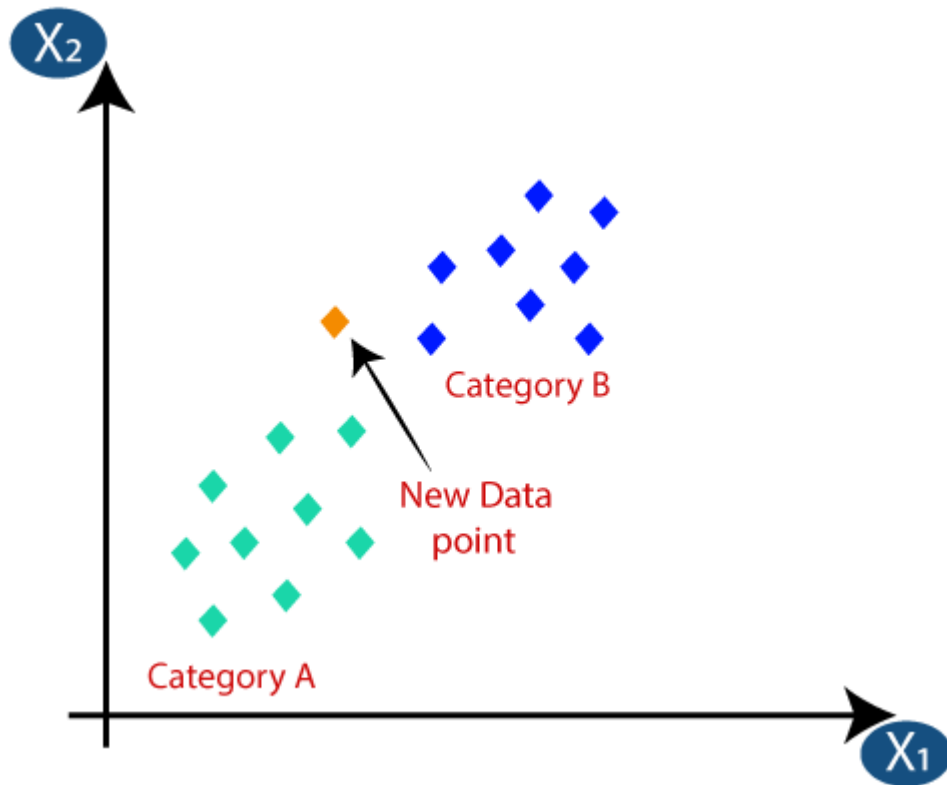
How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.

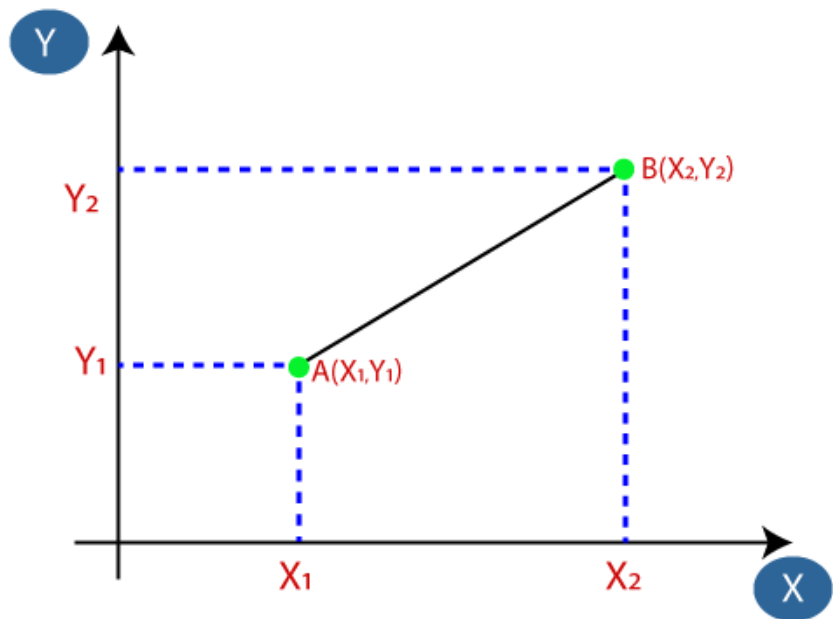
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

Suppose we have a new data point and we need to put it in the required category. Consider the below image:



- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry.

It can be calculated as:



$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

Advantages of KNN Algorithm:

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples

Advantages of Supervised learning:

- With the help of supervised learning, the model can predict the output on the basis of prior experiences.
- In supervised learning, we can have an exact idea about the classes of objects.
- Supervised learning model helps us to solve various real-world problems such as **fraud detection, spam filtering**, etc.

Disadvantages of supervised learning:

- Supervised learning models are not suitable for handling the complex tasks.
- Supervised learning cannot predict the correct output if the test data is different from the training dataset.
- Training required lots of computation times.
- In supervised learning, we need enough knowledge about the classes of object.

Regression:

1. Introduction

Regression analysis is one of the most important statistical techniques used to study the relationship between variables. It helps in predicting the value of one variable based on another variable.

Definition

Regression analysis is a statistical method used to determine the relationship between a **dependent variable (Y)** and one or more **independent variables (X)** for prediction and decision-making.

Objectives of Regression Analysis

- Predict future values.
 - Measure the relationship between variables.
 - Identify significant factors affecting an outcome.
 - Support business and scientific decision-making.
 - Forecast trends.
-

2. Terminology

Term	Meaning
Dependent Variable (Y)	Variable to be predicted (Output)
Independent Variable (X)	Variable used for prediction (Input)
Coefficient	Amount of change in Y for one unit change in X
Intercept	Value of Y when $X = 0$
Regression Line	Best-fit line showing the relationship

3. Types of Regression Analysis

1. Linear Regression

Relationship is represented by a straight line.

Example:

- Salary prediction from experience.

2. Multiple Linear Regression

Uses more than one independent variable.

Example:

- House price prediction using area, location, and age.

3. Polynomial Regression

Models curved relationships.

Example:

- Growth of plants over time.

4. Logistic Regression

Used when the output is categorical.

Example:

- Email Spam Detection (Spam/Not Spam)

5. Ridge Regression

Handles multicollinearity by adding a penalty term.

Example:

- Stock market prediction.

6. Lasso Regression

Performs variable selection and reduces unnecessary features.

Example:

- Medical diagnosis.

7. Elastic Net Regression

Combination of Ridge and Lasso Regression.

8. Support Vector Regression (SVR)

Used for predicting continuous values with high accuracy.

Example:

- Weather forecasting.
-

9. Decision Tree Regression

Uses a tree structure for prediction.

Example:

- Sales forecasting.
-

10. Random Forest Regression

Combines multiple decision trees to improve prediction.

Example:

- Crop yield prediction.
-

4. Real-World Applications of Regression Analysis

Field	Application
Real Estate	Predicting house prices
Education	Predicting student marks
Banking	Loan risk analysis
Healthcare	Predicting medical expenses

Field	Application
Agriculture	Crop yield prediction
Marketing	Sales forecasting
Manufacturing	Demand prediction
Finance	Stock price forecasting
Insurance	Premium calculation
Weather	Rainfall prediction

Linear Regression

Definition

Linear regression is a statistical technique that models the relationship between variables using a straight line.

It predicts the value of a dependent variable from one or more independent variables.

Types of Linear Regression

1. Simple Linear Regression

Uses **one** independent variable.

Equation

$$Y = a + bX$$

Example

Predicting salary using years of experience.

2. Multiple Linear Regression

Uses **two or more** independent variables.

Equation

$$Y = a + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

$$Y = a + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Example

Predicting house price using:

- Area
- Location
- Number of bedrooms
- Age of house

Components of Linear Regression

Symbol	Meaning
Y	Dependent Variable
X	Independent Variable
a	Intercept
b	Regression Coefficient (Slope)

Mathematical Equation

Simple Linear Regression

$$Y = a + bX$$

Where

- **Y** = Predicted value
- **X** = Independent variable
- **a** = Intercept
- **b** = Slope of regression line

Interpretation of Slope

Positive Slope ($b > 0$)

As X increases, Y increases.

Example:

Study Hours $\uparrow \rightarrow$ Marks $\uparrow$

Negative Slope ($b < 0$)

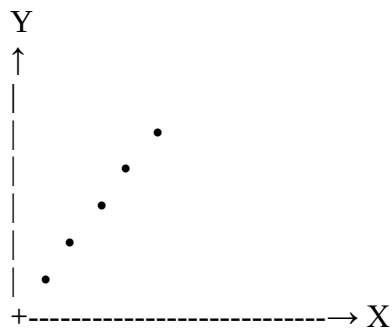
As X increases, Y decreases.

Example:

Speed $\uparrow \rightarrow$ Time Taken $\downarrow$

Regression Line Diagrams

Positive Linear Regression



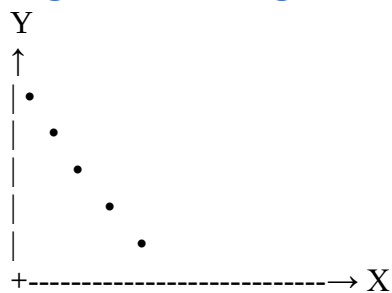
Characteristics

- Upward slope
- Positive relationship
- Both variables increase together

Example:

Advertising Cost $\uparrow \rightarrow$ Sales $\uparrow$

Negative Linear Regression



Characteristics

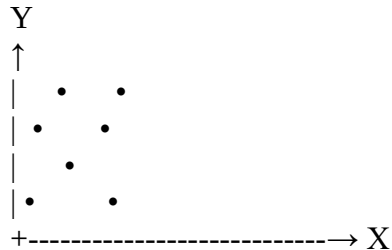
- Downward slope
- Negative relationship

- One variable increases while the other decreases

Example:

Product Price $\uparrow \rightarrow$ Demand $\downarrow$

Zero Relationship



Characteristics

- No trend
- Variables are unrelated

Steps in Regression Analysis

1. Define the problem.
2. Collect data.
3. Choose dependent and independent variables.
4. Plot the data.
5. Fit the regression model.
6. Estimate regression coefficients.
7. Evaluate model accuracy (e.g., R^2 , Mean Squared Error).
8. Make predictions.
9. Interpret the results.

Advantages

- Simple and easy to understand.
 - Predicts future values.
 - Helps in business planning.
 - Supports decision-making.
 - Measures relationships between variables.
 - Widely used across industries.
-

Limitations

- Assumes a linear relationship.
 - Sensitive to outliers.
 - Requires quality data.
 - Correlation does not imply causation.
 - Prediction accuracy depends on the model.
-

Exam-Oriented Short Notes

Regression Analysis

- Statistical technique for prediction.
- Studies relationship between dependent and independent variables.
- Used in forecasting and decision-making.

Linear Regression

- Represents the relationship using a straight line.
- Two types:
 - Simple Linear Regression
 - Multiple Linear Regression

Formula

$$Y = a + bX$$

Where:

- **Y** = Dependent variable
 - **X** = Independent variable
 - **a** = Intercept
 - **b** = Slope
-

UNIT 5: Unsupervised Machine Learning Models

What is Unsupervised Learning?

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



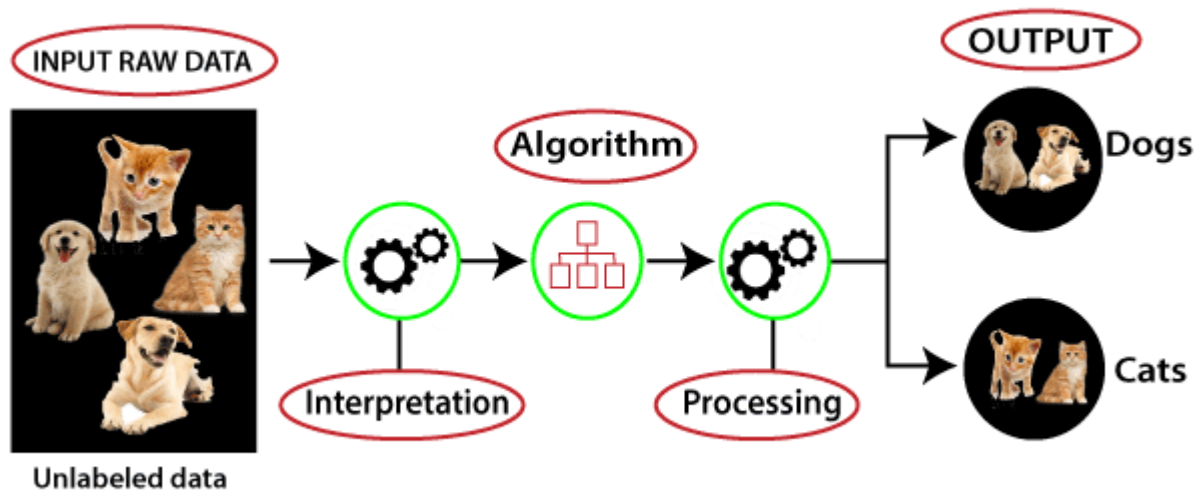
Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:

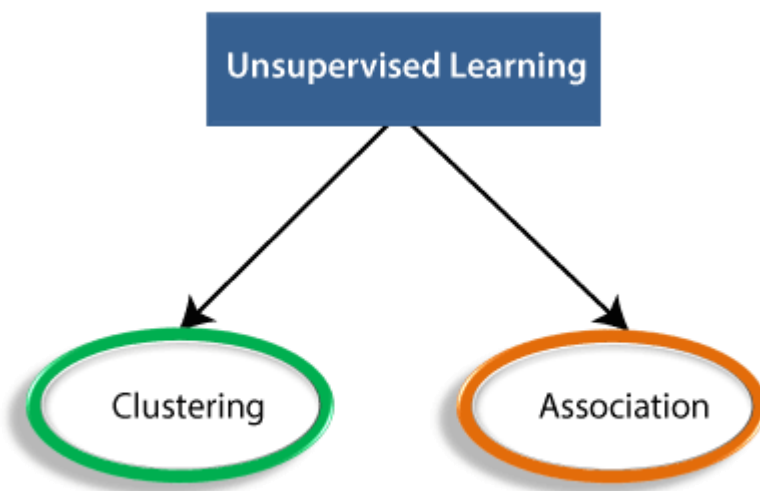


Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain in a group and have less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.

Types of Clustering Methods

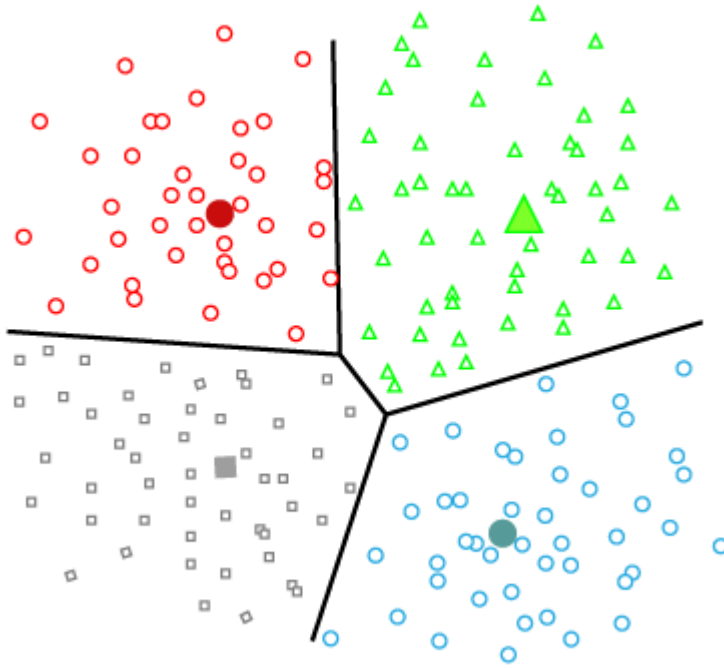
The clustering methods are broadly divided into **Hard clustering** (datapoint belongs to only one group) and **Soft Clustering** (data points can belong to another group also). But there are also other various approaches of Clustering exist. Below are the main clustering methods used in Machine learning:

1. **Partitioning Clustering**
2. **Density-Based Clustering**
3. **Distribution Model-Based Clustering**
4. **Hierarchical Clustering**
5. **Fuzzy Clustering**

Partitioning Clustering

It is a type of clustering that divides the data into non-hierarchical groups. It is also known as the **centroid-based method**. The most common example of partitioning clustering is the **K-Means Clustering algorithm**.

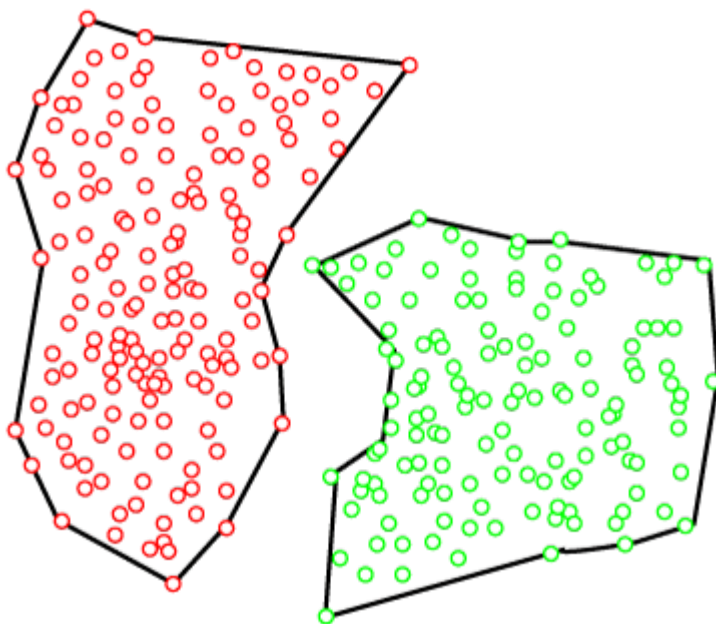
In this type, the dataset is divided into a set of k groups, where K is used to define the number of pre-defined groups. The cluster center is created in such a way that the distance between the data points of one cluster is minimum as compared to another cluster centroid.



Density-Based Clustering

The density-based clustering method connects the highly-dense areas into clusters, and the arbitrarily shaped distributions are formed as long as the dense region can be connected. This algorithm does it by identifying different clusters in the dataset and connects the areas of high densities into clusters. The dense areas in data space are divided from each other by sparser areas.

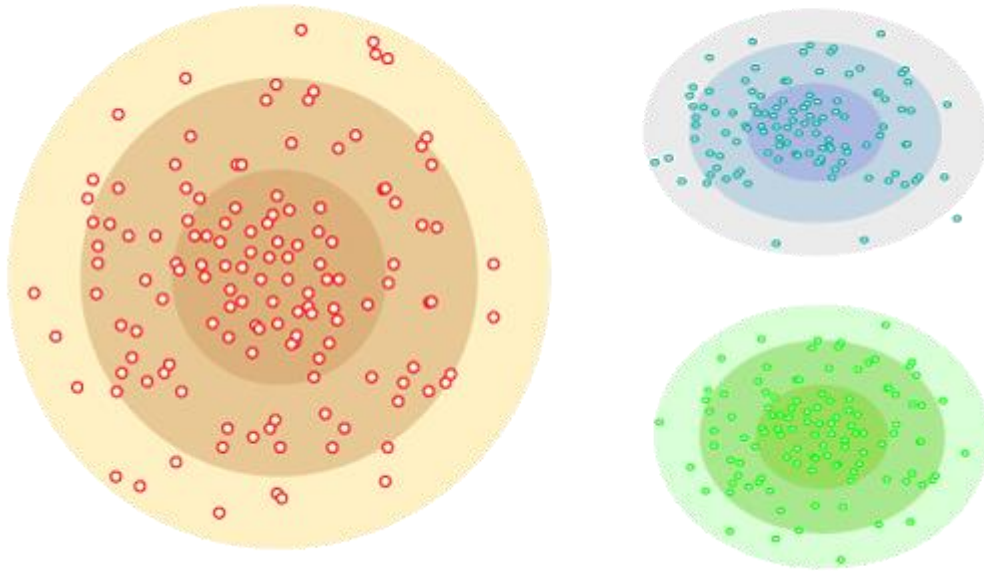
These algorithms can face difficulty in clustering the data points if the dataset has varying densities and high dimensions.



Distribution Model-Based Clustering

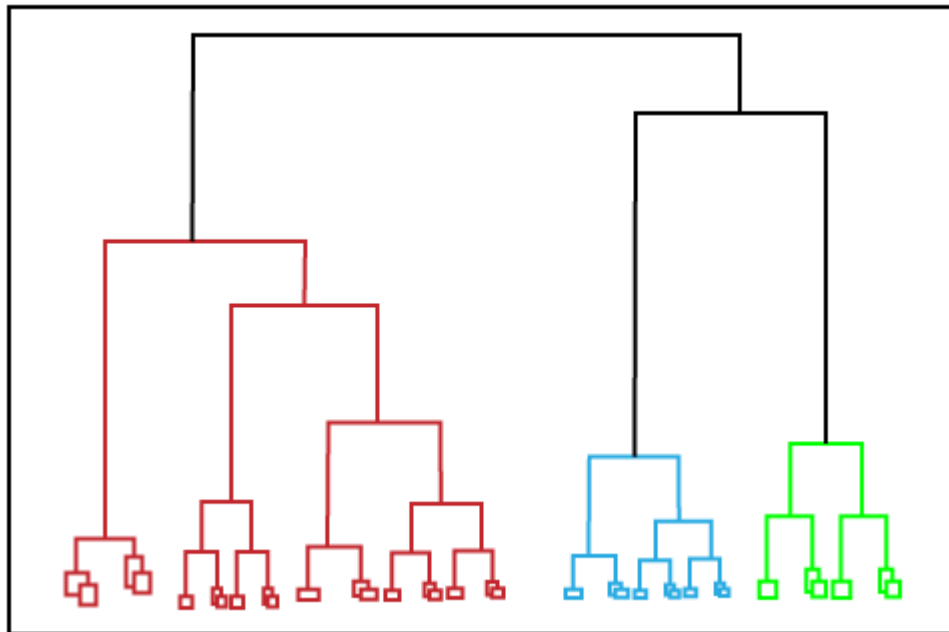
In the distribution model-based clustering method, the data is divided based on the probability of how a dataset belongs to a particular distribution. The grouping is done by assuming some distributions commonly **Gaussian Distribution**.

The example of this type is the **Expectation-Maximization Clustering algorithm** that uses Gaussian Mixture Models (GMM).



Hierarchical Clustering

Hierarchical clustering can be used as an alternative for the partitioned clustering as there is no requirement of pre-specifying the number of clusters to be created. In this technique, the dataset is divided into clusters to create a tree-like structure, which is also called a **dendrogram**. The observations or any number of clusters can be selected by cutting the tree at the correct level. The most common example of this method is the **Agglomerative Hierarchical algorithm**.



Fuzzy Clustering

Fuzzy clustering is a type of soft method in which a data object may belong to more than one group or cluster. Each dataset has a set of membership coefficients, which depend on the degree of membership to be in a cluster. **Fuzzy C-means algorithm** is the example of this type of clustering; it is sometimes also known as the Fuzzy k-means algorithm.

- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Association Rule Learning

Association rule learning is a type of unsupervised learning technique that checks for the dependency of one data item on another data item and maps accordingly so that it can be more profitable. It tries to find some interesting relations or associations among the variables of dataset. It is based on different rules to discover the interesting relations between variables in the database.

The association rule learning is one of the very important concepts of machine learning, and it is employed in **Market Basket analysis, Web usage mining, continuous production, etc.** Here market basket analysis is a technique used by the various big retailer to discover the

associations between items. We can understand it by taking an example of a supermarket, as in a supermarket, all products that are purchased together are put together.

For example, if a customer buys bread, he most likely can also buy butter, eggs, or milk, so these products are stored within a shelf or mostly nearby. Consider the below diagram:



Association rule learning can be divided into three types of algorithms:

1. **Apriori**
2. **Eclat**
3. **F-P Growth Algorithm**

How does Association Rule Learning work?

Association rule learning works on the concept of If and Else Statement, such as if A then B.



Here the If element is called **antecedent**, and then statement is called as **Consequent**. These types of relationships where we can find out some association or relation between two items is known as *single cardinality*. It is all about creating rules, and if the number of items increases, then cardinality also increases accordingly. So, to measure the associations between thousands of data items, there are several metrics. These metrics are given below:

- **Support**
- **Confidence**
- **Lift**

Let's understand each of them:

Support

Support is the frequency of A or how frequently an item appears in the dataset. It is defined as the fraction of the transaction T that contains the itemset X. If there are X datasets, then for transactions T, it can be written as:

$$\text{Supp}(X) = \frac{\text{Freq}(X)}{T}$$

Confidence

Confidence indicates how often the rule has been found to be true. Or how often the items X and Y occur together in the dataset when the occurrence of X is already given. It is the ratio of the transaction that contains X and Y to the number of records that contain X.

$$\text{Confidence} = \frac{\text{Freq}(X,Y)}{\text{Freq}(X)}$$

Lift

It is the strength of any rule, which can be defined as below formula:

$$\text{Lift} = \frac{\text{Supp}(X,Y)}{\text{Supp}(X) \times \text{Supp}(Y)}$$

It is the ratio of the observed support measure and expected support if X and Y are independent of each other. It has three possible values:

- If **Lift= 1**: The probability of occurrence of antecedent and consequent is independent of each other.
- **Lift>1**: It determines the degree to which the two itemsets are dependent to each other.
- **Lift<1**: It tells us that one item is a substitute for other items, which means one item has a negative effect on another.

Types of Association Rule Learning

Association rule learning can be divided into three algorithms:

Apriori Algorithm

This algorithm uses frequent datasets to generate association rules. It is designed to work on the databases that contain transactions. This algorithm uses a breadth-first search and Hash Tree to calculate the itemset efficiently.

It is mainly used for market basket analysis and helps to understand the products that can be bought together. It can also be used in the healthcare field to find drug reactions for patients.

Eclat Algorithm

Eclat algorithm stands for **Equivalence Class Transformation**. This algorithm uses a depth-first search technique to find frequent itemsets in a transaction database. It performs faster execution than Apriori Algorithm.

F-P Growth Algorithm

The F-P growth algorithm stands for **Frequent Pattern**, and it is the improved version of the Apriori Algorithm. It represents the database in the form of a tree structure that is known as a frequent pattern or tree. The purpose of this frequent tree is to extract the most frequent patterns.

Applications of Association Rule Learning

It has various applications in machine learning and data mining. Below are some popular applications of association rule learning:

- **Market Basket Analysis:** It is one of the popular examples and applications of association rule mining. This technique is commonly used by big retailers to determine the association between items.
- **Medical Diagnosis:** With the help of association rules, patients can be cured easily, as it helps in identifying the probability of illness for a particular disease.
- **Protein Sequence:** The association rules help in determining the synthesis of artificial Proteins.
- It is also used for the **Catalog Design** and **Loss-leader Analysis** and many more other applications.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- **K-means clustering**
- **KNN (k-nearest neighbors)**
- **Hierarchical clustering**
- **Anomaly detection**
- **Neural Networks**
- **Principle Component Analysis**
- **Independent Component Analysis**
- **Apriori algorithm**
- **Singular value decomposition**

Advantages of Unsupervised Learning

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

Real world examples of unsupervised Learning

Unsupervised machine learning is the process of inferring underlying hidden patterns from historical data. It finds a myriad of real-life applications, including:

1. **Data Exploration:** Unsupervised learning can be used to explore the data and find some structure within it
2. **Customer Segmentation:** It can be used to segment customers into different groups based on their purchasing behavior, demographics, etc.
3. **Recommender Systems:** Websites like Amazon use unsupervised learning algorithms to implement user-specific recommendation systems.
4. **Target Marketing Campaigns:** Businesses can use unsupervised learning to target specific groups of customers for marketing campaigns
5. **Data Preparation and Visualization:** Unsupervised learning can be used for effective data preparation and visualization.
6. **Anomaly Detection:** It can be used to detect anomalies or outliers in the data.
7. **Dimensionality Reduction:** Unsupervised learning can be used to reduce the number of features in a dataset while retaining the essential information.

For example, in the case of a YouTube channel, if you want to detect groups of similar subscribers, you may need to run a clustering algorithm. The algorithm can find those connections without your help. For example, it may tell you that 35% of your subscribers are from Canada, while 20% of them are from the United States

Differentiate Supervised and Unsupervised Learning

Supervised Learning:

- In supervised learning, the model is trained on a labeled dataset. That is, each example in the training dataset is associated with a label or output value.
- The goal of a supervised learning algorithm is to learn a function that, given a sample of data and desired outputs, best approximates the relationship between input and output observable in the data.
- Once the model is trained, it can be used to predict the output for new or unseen data.
- Examples of supervised learning algorithms include linear regression, logistic regression, support vector machines, and neural networks.

Unsupervised Learning:

- In contrast to supervised learning, unsupervised learning algorithms are trained on datasets without labels. The goal here is to infer the natural structure present within a set of data points.
- Unsupervised learning can be a goal in itself (discovering hidden patterns in data) or a means towards an end (feature learning).
- Examples of unsupervised learning algorithms include k-means clustering, hierarchical clustering, and Principal Component Analysis (PCA).

